

Python

Notebooks, scripts and images from Python.zip

September 06, 2026

Contents

Notebooks

- [arraymodule.ipynb](#)
- [B.tech_lab.ipynb](#)
- [Class.ipynb](#)
- [Class2.ipynb](#)
- [class3.ipynb](#)
- [data_structure.ipynb](#)
- [django.ipynb](#)
- [function.ipynb](#)
- [function_practice.ipynb](#)
- [Matrix.ipynb](#)
- [sql.ipynb](#)
- [Untitled.ipynb](#)

Scripts

- [cal.py](#)
- [calc.py](#)
- [client.py](#)
- [dem.py](#)
- [server.py](#)
- [sql.py](#)

Images

- [new.png](#)
- [new1.png](#)
- [q13.png](#)
- [q15.png](#)
- [q8.jpeg](#)
- [q9.jpeg](#)

arraymodule.ipynb

In [1]

```
from array import*
vals= array('i', [1,2,3,4])
new=array(vals.typecode, (a*a for a in vals))
for i in vals:
    print(i)
i=0
while i<len(new):
    print (new[i])
    i=i+1
```

Out

```
1
2
3
4
1
4
9
16
```

In [2]

```
from array import*
arr=array('i',[])
n=int(input('Enter the length of array'))
for i in range(n):
    x=int(input('Enter the next element of array'))
    arr.append(x)
print(arr)
```

In [3]

```
for i in a:
```

In [4]

```
val=int(input('Enter the value for search'))
k=0
for e in arr:
    if e ==val:
        print(k)
        break
    k=k+1
#array.index(val)
```

Out

```
Enter the value for search7
1
```

In [5]

```
from array import*
vals= array('i', [[1,2,3,4],[2,3,4,6]])
```

Out

```
-----
TypeError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_6044\3609360141.py in <module>
      1 from array import*
----> 2 vals= array('i', [[1,2,3,4],[2,3,4,6]])

TypeError: an integer is required (got type list)
```

In [6]

```
#from array import*
#vals= array('i', [[1,2,3,4],[2,3,4,6]])
from numpy import*
arr= array([[1,2,3,4],[2,3,4,6]])
#if put different data type conveted into same
#print(arr.dtype)
```

In [7]

```
arr= ones(10)
print(arr)
```

Out

```
[1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

In [8]

```
arr= linspace(start, end, parts)
arr= logspace(start, end, parts)
arr= arange(start, end, step)
arr= zeros(number)
arr= ones()
```

In [9]

```
from numpy import*
arr1=array([1,6,13,3])
arr2=arr1+5
arr3=arr1+arr2
print(sin(arr1))
#min, max, sum, unique,sort, concanetate two array
```

Out

```
[ 0.84147098 -0.2794155  0.42016704  0.14112001]
```

In [10]

```
from numpy import*
arr1=array([1,6,13,3])
arr2=arr1
print(id(arr1))
print(id(arr2))
print(arr1)
print(arr2)
```

Out

```
2697927394768
2697927394768
[ 1  6 13  3]
[ 1  6 13  3]
```

In [11]

```
from numpy import*
arr1=array([1,6,13,3])
arr2=arr1.view()
arr1[1]=7
#shallow copy
print(id(arr1))
print(id(arr2))
print(arr1)
print(arr2)
```

Out

```
2697949537552
2697949537840
[ 1  7 13  3]
[ 1  7 13  3]
```

In [12]

```
from numpy import*
arr1=array([1,6,13,3])
arr2=arr1.copy()
arr1[1]=7
#deep copy
print(id(arr1))
print(id(arr2))
print(arr1)
print(arr2)
```

Out

```
2697928237968
2697949537744
[ 1  7 13  3]
[ 1  6 13  3]
```

In [13]

```
from numpy import*
arr= array([[1,2,3],[4,5,6]])
print(arr)
print(arr.shape)
print(arr.size)
print(arr.dtype)
print(arr.ndim)
arr2=arr.flatten()
print(arr2)
```

Out

```
[[1 2 3]
 [4 5 6]]
(2, 3)
6
int32
2
[1 2 3 4 5 6]
```

In [14]

```
from numpy import*
arr3= array([[1,2,3,8,32,5],[4,5,6,43,21,1]])
print(arr3)
arr2=arr3.flatten()
print(arr2)
#arr2.reshape(3,4)
arr2.reshape(2,2,3)
```

Out

```
[[ 1  2  3  8 32  5]
 [ 4  5  6 43 21  1]]
[ 1  2  3  8 32  5  4  5  6 43 21  1]

array([[[ 1,  2,  3],
        [ 8, 32,  5]],

       [[ 4,  5,  6],
        [43, 21,  1]]])
```

In [15]

```
from numpy import*
arr3= array([[1,2,3,8],[4,5,6,43]])
m=matrix(arr3)
m1=matrix('1,2,3; 4,5,6; 7,8,9')
m2=matrix('1,2,3; 4,5,6; 7,8,9')
m1+m2
m1*m2
print(diagonal(m))
print(m.max())
```

Out

```
[1 5]
43
```

In [16]

```
#for else
nums=[12,16,18,21,26]
for num in nums:
    if num%5==0:
        print(num)
        break
##else in if part then it is printed multiple times
else:
    print('notfound')
```

Out

```
notfound
```

In [17]

```
#pass
def greet():
    pass
#skip the block
for i in range(1,101):
    if i%3 or i%5==0:
        continue
    print(i)
print('Bye')

for i in range(1,101):
    if i%2!=0:
        pass
    else:
        print(i)
```

Out

```
3
6
9
12
18
21
24
27
33
36
39
42
48
51
54
57
63
66
69
72
78
81
84
```

87
93
96
99
Bye
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
32
34
36
38
40
42
44
46
48
50
52
54
56
58
60
62
64
66
68
70
72
74
76
78
80
82
84
86
88
90
92
94
96
98
100

In [18]

```
for i in range(1,101):
    if i%2!=0:
        pass
    else:
        print(i)
```

Out

```
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
32
34
36
38
40
42
44
46
48
50
52
54
56
58
60
62
64
66
68
70
72
74
76
78
80
82
84
86
88
90
92
94
96
98
100
```

In [19]

```
#prime
num=7
for i in range(2,num):
    if num%i==0:
        print('not prime')
        break
    else:
        print('prime')
```

Out

```
prime
prime
prime
prime
prime
```

In [20]

```
#pattern
print('# # # #')
```

Out

```
# # # #
```

In [21]

```
print('#')
print('#')
print('#')
print('#')
```

Out

```
#
#
#
#
```

In [22]

```
print('#', end='')
print('#', end='')
print('#', end='')
print('#',end='')
```

Out

```
####
```

In [23]

```
for j in range(4):
    print('#', end='')
```

Out

```
####
```

In [24]

```
for i in range(4):
    for j in range(4):
        print('#', end='')
    print()
```

Out

```
####
####
####
####
```

In [25]

```
for i in range(4):
    for j in range(i):
        print('#', end='')
    print()
```

Out

```
#
##
###
```

In [26]

```
#print VIT BHOPAL
```

In [27]

```
import keyword

# printing all keywords at once using "kwlist()"
print("The list of keywords is : ")
print(keyword.kwlist)
```

Out

```
The list of keywords is :
['False', 'None', 'True', '__peg_parser__', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else',
'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try',
'while', 'with', 'yield']
```

B.tech_lab.ipynb

In [1]

```
#This is my first python program
```

In [2]

```
print("hi")
```

Out

```
hi
```

In [3]

```
"""
print("hi")
Jupyter:IPython notebook ("interactive python").
Spyder : Integrated Development Environment (IDE)
"""
```

Out

```
'\nprint("hi")\nJupyter:IPython notebook ("interactive python").\nSpyder : Integrated Development Environment (IDE) \n'
```

In [4]

```
#Dynamic typed language
x = 5
y = "John"
print(x)
print(y)
```

Out

```
5
John
```

In [5]

```
#Python has the following data types built-in by default
#list,tuple,dict,set
```

In [6]

```
#list is a data structure that's built into Python and holds a collection of items
#Lists are ordered – i.e. the items in the list appear in a specific order. This enables us to use an index to access to any item.
#Lists are mutable, which means you can add or remove items after a list's creation.
#do not need to be unique
#different data types
```

In [7]

```
list = [3, 6, 9,"hi", 12]
print(list)
print(type(list))
```

Out

```
[3, 6, 9, 'hi', 12]
<class 'list'>
```

In [8]

```
#array module requires all array elements to be of the same type
#you'll need to specify a value type.
import array as arr
array_1 = arr.array("i", [3, 6, 9, 12])
print(array_1)
print(type(array_1))
```

Out

```
array('i', [3, 6, 9, 12])
<class 'array.array'>
```

In [9]

```
#NumPy arrays support different data types.
#To create a NumPy array, you only need to specify the items
import numpy as np
array_2 = np.array([ 3, 6, 9, 12])
print (array_2)
print(type(array_2))
```

Out

```
[ 3  6  9 12]
<class 'numpy.ndarray'>
```

In [10]

```
#Arrays need to be declared. Lists don't, since they are built into Python.
#Arrays can store data very compactly and are more efficient for storing large amounts of data.
#Arrays are great for numerical operations; lists cannot directly handle math operations.
```

In [11]

```
array = np.array([3, 6, 9, 12])
division = array/3
print(division)
print (type(division))
```

Out

```
[1.  2.  3.  4.]
<class 'numpy.ndarray'>
```

In [12]

```
list = [3, 6, 9, 12]
division = list/3
```

Out

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-11-f127235414b2> in <module>()
      1 list = [3, 6, 9, 12]
----> 2 division = list/3

TypeError: unsupported operand type(s) for /: 'list' and 'int'
```

In [13]

```
#Dictionary
#Unordered list
#key:value accociation
#key cannot be mutable
empty = {}
```

In [14]

```
food = {"ham" : "yes", "egg" : "yes", "spam" : "no" }
print(food)
```

Out

```
{'ham': 'yes', 'egg': 'yes', 'spam': 'no'}
```

In [15]

```
food["spam"] = "yes"
print(food)
```

Out

```
{'ham': 'yes', 'egg': 'yes', 'spam': 'yes'}
```

In [16]

```
words = {"house" : "Haus", "cat":"Katze"}
#words["car"]
```

In [17]

```
# INDENTATION MATTERS
# To define a block, either,if block, for block, function block, class block
if "car" in words:
    print (words["car"])
```

In [18]

```
if "cat" in words:
    print (words["cat"])
```

Out

```
Katze
```

In [19]

```
for key in words:
    print (key)
```

Out

```
house
cat
```

In [20]

```
#Tuple
#Tuple is often compared with List as it looks very much like a list
#Tuple is considered to be the fastest and they consume the least amount of memory.
# A tuple is immutable.
#If you really need to use a static array in your program,
#you should use tuple as it is better at memory management than an array
#and does the same job as a static array.
```

In [21]

```
z = float("3")
print(z)
```

Out

```
3.0
```

In [22]

```
thistuple = ("apple", "banana", "cherry")
print(thistuple[-1])
```

Out

```
cherry
```

In [23]

```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(thistuple[-4:-2])
```

Out

```
('orange', 'kiwi')
```

In [24]

```
thistuple = ("apple", "banana", "cherry")
thistuple[2] = "orange" # This will raise an error
print(thistuple)
```

Out

```
-----
TypeError                                 Traceback (most recent call last)
<ipython-input-22-13df36b3dac6> in <module>()
      1 thistuple = ("apple", "banana", "cherry")
----> 2 thistuple[2] = "orange" # This will raise an error
      3 print(thistuple)

TypeError: 'tuple' object does not support item assignment
```

In [25]

```
#List is a collection which is ordered and changeable. Allows duplicate members.
#Tuple is a collection which is ordered and unchangeable. Allows duplicate members.
#Set is a collection which is unordered and unindexed. No duplicate members.
#Dictionary is a collection which is unordered, changeable and indexed. No duplicate members.
```

In [26]

```
#if_else statement
a = 33
b = 33
if b > a:
    print("b is greater than a")
elif a == b:
    print("a and b are equal")
```

Out

```
a and b are equal
```

In [27]

```
#while_loop
i = 1
while i < 6:
    print(i)
    i += 1
```

Out

```
1
2
3
4
5
```

In [28]

```
def my_function():
    print("Hello from a function")

my_function()
```

Out

```
Hello from a function
```

In [29]

```
#Return multiple value in one function
def double_return(x):
    return 5 * x,x
```

In [30]

```
double_return(x)
```

Out

```
(25, 5)
```

In [31]

```
#A lambda function is a small anonymous function.
#A lambda function can take any number of arguments, but can only have one expression.
#lambda arguments : expression
x = lambda a : a + 10
print(x(5))
```

Out

```
15
```

In [32]

```
#DEFINE an array
cars = ["Ford", "Volvo", "BMW"]
```

In [33]

```
# Using above first method to create a
# 2D array
rows, cols = (5, 5)
arr = [[0]*cols]*rows
print(arr)
```

Out

```
[[0, 0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0, 0, 0]]
```

In [34]

```
# A basic code for matrix input from user

R = int(input("Enter the number of rows:"))
C = int(input("Enter the number of columns:"))

# Initialize matrix
matrix = []
print("Enter the entries rowwise:")

# For user input
for i in range(R):          # A for loop for row entries
    a=[]
    for j in range(C):      # A for loop for column entries
        a.append(int(input()))
    matrix.append(a)

# For printing the matrix
for i in range(R):
    for j in range(C):
        print(matrix[i][j], end = " ")
    print()
```

In [35]

```
#class in python
class MyClass:
    x = 5
```

In [36]

```
p1 = MyClass()
print(p1.x)
```

In [37]

```
#Create a class named Person, use the __init__() function to assign values for name and age:
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("John", 36)

print(p1.name)
print(p1.age)
```

In [38]

```
#Objects can also contain methods.
#Methods in objects are functions that belong to the object.
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)

p1 = Person("John", 36)
p1.myfunc()
```

Out

```
Hello my name is John
```

In [39]

```
#Consider a module to be the same as a code library.
#A file containing a set of functions you want to include in your application.
#Save this code in a file named mymodule.py
def greeting(name):
    print("Hello, " + name)
```

In [40]

```
#Now we can use the module we just created, by using the import statement:
import mymodule

mymodule.greeting("Jonathan")
```

Class.ipynb

In [1]

```
# Python is a functional, procedural and object oriented programming language
# real world object (attributes (data, variable), behavior (function or methods))
# virtual (software-solving real problem in virtual environment )
# multiple objects(phones) of same type designed(class), manufacture anywhere
# class-design(or blueprint) of object
#objects(instance of a class) are real stuffs
```

In [2]

```
class Computer:
    def config(self):
        print("i5, 16gb, 1TB")

a='a'
print(type(a))
com1=Computer()
com2=Computer()
print(type(com1))
#pass object
#self is a object that we pass

Computer.config(com2)
# more popular way of calling function of a class
com1.config()
com2.config()
a=10
a.
```

Out

```
<class 'str'>
<class '__main__.Computer'>
i5, 16gb, 1TB
i5, 16gb, 1TB
i5, 16gb, 1TB
i5, 16gb, 1TB

4
```

In [3]

```
class Computer:
    #init method called automatically
    def __init__(self):#constructor
        print("in init")
    def config(self):
        print("i5, 16gb, 1TB")

com1=Computer()
com2=Computer()
com1.config()
com2.config()
```

Out

```
in init
in init
i5, 16gb, 1TB
i5, 16gb, 1TB
```

In [4]

```
class Computer:
    #init method called automatically
    def __init__(self, cpu, ram):
        self.cpu=cpu
        self.ram=ram
    def config(self):
        print("config is:", self.cpu, self.ram)

com1=Computer('i5', 16)
com2=Computer('i3', 8)
com1.config()
com2.config()
#methdod and data work together
#binding our data with every method
#one object has its own variable and method
```

Out

```
config is: i5 16
config is: i3 8
```

In [5]

```
#init is actually a constructor
#every objects take space in heap memory
class Computer:
    def __init__(self):
        self.name="fahad"
        self.age=30
    pass
c1=Computer()
print(c1.name)
c2=Computer()
print(c2.name)
#every time you create an object it is allocated to new space
#size of object depends on the number of variable
#who allocate the size to object- answer is constructor, the init method will be called implicitly
```

Out

```
fahad
fahad
```

In [6]

```
class Computer:
    def __init__(self):
        self.name="fahad"
        self.age=30
    pass
    def update(self):
        self.age=19
c1=Computer()
c1.name="jayesh"
c1.age=18
c1.update()
# self refers to the current object
print(c1.age)
```

Out

```
19
```

In [7]

```
class Computer:
    def __init__(self):
        self.name="fahad"
        self.age=30

    def compare(self, other):
        if self.age==other.age:
            return True
        else:
            return False

c1=Computer()
c2=Computer()
c1.age=20
if c1.compare(c2):
    print("They are same")
else:
    print("They are different")

#comparing two objects
```

Out

```
They are different
```

In [8]

```
#two different types of variables in oops
# instance variable, class(static) variables
class car:
    wheels=5# variable outside the init method
    def __init__(self):
        self.mil=10 #variable inside the init method
        self.com="BM"

c1=car()
c2=car()
c1.mil=8
car.wheels=4
print(c1.com,c1.mil, c1.wheels)
print(c2.com,c2.mil, c2.wheels)
#Static variables can also be called by class name
#in memory name space is an area where we create and store object/variable
#(1) class namespace
#(2)object/instance namespace
```

Out

```
BM 8 4
BM 10 4
```

In [9]

```
#Types of methods in python
#instance method, class methods, static methods
class student:
    school='VIT'
    def __init__(self, m1,m2,m3):
        self.m1=m1
        self.m2=m2
        self.m3=m3
    def avg(self):#instance method
        return (self.m1+self.m2+self.m3)/3
    def get_m1(self):#Accessor
        return self.m1
    def set_m1(self, value):#Mutator
        self.m1=value
s1=student(36,67,32)
s2=student=(89,32,12)
print(s1.avg())
#instance method is of two ypes
#Accessor and mutator
```

Out

```
45.0
```

In [10]

```
class student:
    school='VIT'
    def __init__(self, m1,m2,m3):
        self.m1=m1
        self.m2=m2
        self.m3=m3
    def avg(self):#instance method
        return (self.m1+self.m2+self.m3)/3
    @classmethod #decorators
    def get_school (cls):#classmethod
        return cls.school

    # @staticmethod #decorators
    def info ():#neither related to instance variable nor related to class variable
        #dont pass anything#uses for other class object, factorial of a number
        print("This is student class in ABC module")
print (student.get_school())
print (student.info())
```

Out

```
VIT
This is student class in ABC module
None
```

In [11]

```
#class inside class
class student:#outsider class
    def __init__(self,name,rollno):
        self.name=name
        self.rollno=rollno
        self.lap=self.Laptop()
    def show(self):
        print(self.name,self.rollno)
        self.lap.show()
    class Laptop:#inner class
        def __init__(self):
            self.brand='HP'
            self.cpu="i5"
            self.ram=8
        def show(self):
            print(self.brand,self.cpu,self.ram)
s1=student('Navin', 2)
s2=student('Jenny', 3)
s1.show()
#s1.lap.brand
#or
lap1=s1.lap
lap2=s2.lap
print(id(lap1))
print(id(lap2))
#you can create object of inner class inside the outer class or
#you can create object of inner class outside the outer class provided you use outer class name to call it
lap1=student.Laptop()
```

Out

```
Navin 2
HP i5 8
2017845896960
2017845899024
```

In [12]

```
#inheritance

class A:#super class or parent class
    def feature1(self):
        print("Feature1 working")
    def feature2(self):
        print("Feature2 working")
class B:
    def feature3(self):
        print("Feature3 working")
    def feature4(self):
        print("Feature4 working")
#a1=A()
#a1.feature1()
#a1.feature2()
#b1=B()
#b1.feature3()
#b1.feature4()
class B(A):#sub class or child class # single level inheritance
    def feature3(self):
        print("Feature3 working")
    def feature4(self):
        print("Feature4 working")
class C(B):#multilevel inheritance
    def feature5(self):
        print("Feature5 working")

b1=B()
c1=C()
b1.feature1()
```

Out

```
Feature1 working
```

In [13]

```
class A:#Multiple inheritance
    def feature1(self):
        print("Feature1 working")
    def feature2(self):
        print("Feature2 working")
class B:
    def feature3(self):
        print("Feature3 working")
    def feature4(self):
        print("Feature4 working")
class C(A,B):#multilevel inheritance
    def feature5(self):
        print("Feature5 working")

c1=C()
c1.feature1()
c1.feature3()
```

Out

```
Feature1 working
Feature3 working
```

In [14]

```
#constructor in inheritance
class A:
    def __init__(self):
        print('Print A in init')

    def feature1(self):
        print("Feature1 working")
    def feature2(self):
        print("Feature2 working")
class B(A):
    def __init__(self):
        super().__init__()#when we want to access the init function of A also
        print('Print B in init')
    def feature3(self):
        print("Feature3 working")
    def feature4(self):
        print("Feature4 working")

a1=A()
b1=B()# when class B has its own charecter then it will not call the constructor of A otherwise does
```

Out

```
Print A in init
Print A in init
Print B in init
```

In [15]

```
class A:#Multiple inheritance
    def __init__(self):
        print('Print A in init')
    def feature1(self):
        print("Feature1-A working")
    def feature2(self):
        print("Feature2 working")
class B:
    def __init__(self):
        print('Print B in init')
    def feature1(self):
        print("Feature1-B working")
    def feature4(self):
        print("Feature4 working")
class C(A,B):#multilevel inheritance
    def __init__(self):
        super().__init__()#it will call the init of A (leftone)
        print('Print C in init')
    def feature5(self):
        print("Feature5 working")
    def feat():
        super().feature2()# super method can also be used for other method rather than init

a1=C()
#Method resolution order in function also
a1.feature1()
```

Out

```
Print A in init
Print C in init
Feature1-A working
```

In [16]

```
#Polymorphism in python
# one thing can take multiple form(like human based on situation)
#objects have multiple form
```

In [17]

```
#Duck typing(behaviour of a bird matching with duck) #dynamicbtyped
x=5
x='fahad'
class jupyter:
    def execute(self):
        print('compiling')
        print('running')
class Myeditor:
    def execute(self):
        print('spell check')
        print('convention check')
        print('compiling')
        print('running')
class Laptop:
    def code(self,ide):
        ide.execute()
#ide1=jupyter()
#we can change ide object provided that we have execute method, dont think which class object "ide"is
#ide=Myeditor()
lap1=Laptop()
lap1.code(ide)
```

Out

```
compiling
running
```

In [18]

```
#Duck typing(behaviour of a bird matching with duck) #dynamicbtyped
x=5
x='fahad'
class jupyter:
    def execute(self):
        print('compiling')
        print('running')
class Myeditor:
    def execute(self):
        print('spell check')
        print('convention check')
        print('compiling')
        print('running')
class Laptop:
    def code(self,ide):
        ide.execute()
#ide1=jupyter()
#we can change ide object provided that we have execute method, dont think which class object "ide" is
ide=Myeditor()
lap1=Laptop()
lap1.code(ide)
```

Out

```
spell check
convention check
compiling
running
```

Class2.ipynb

In [1]

```
#operator overloading
a=5
b=6
#b='world'
c=a+b
print(a+b)
print(int.__add__(a,b))
```

Out

```
11
11
```

In [2]

```
#operators are same but operands are differnt (number of arguments, type of arguments)
class student:
    def __init__(self,m1,m2):
        self.m1=m1
        self.m2=m2
    def __add__(self,others):
        m1=self.m1+others.m1
        m2=self.m2+others.m2
        s3=student(m1,m2)
        return s3
    def __gt__(self,others):
        r1=self.m1+self.m2
        r2=others.m1+others.m2
        if r1>r2:
            return True
        else:
            return False

    def __str__(self):
        #return self.m1, self.m2
        return '{} {}'.format(self.m1, self.m2)

s1=student(50,65)
s2=student(69,64)
s3=s1+s2
print(s3.m1)
if s1>s2:
    print('s1 wins')
else:
    print('s2 wins')
a=9
print(a.__str__())
#print(s1.__str__())#built in object address, overriding this method str
print(s1)
```

Out

```
119
s2 wins
9
50 65
```

In [3]

```
a=9
print(a.__str__())
```

Out

```
9
```

In [4]

```
#Method overloading and method overriding
#method overloading is not in python #div(a,b) and div(a,b,c)
#method overriding div(a,b)and div(a,b) but different class (inheritance)
class student:
    def __init__(self,m1,m2):
        self.m1=m1
        self.m2=m2
    def sum(self,a,b,c):
        s=a+b+c
        return s
s1=student(50,68)
print(s1.sum(12,85))
```

Out

```
-----
TypeError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_2440\2347804859.py in <module>
      10         return s
      11 s1=student(50,68)
--> 12 print(s1.sum(12,85))

TypeError: sum() missing 1 required positional argument: 'c'
```

In [5]

```
class student:
    def __init__(self,m1,m2):
        self.m1=m1
        self.m2=m2
    def sum(self,a=None,b=None,c=None):
        s=0
        if a!= None and b!= None and c!= None:
            s=a+b+c
        elif a!= None and b!= None:
            s=a+b
        else:
            s=a
        return s
s1=student(50,68)
print(s1.sum(12,85))
```

Out

97

In [6]

```
class student:
    def __init__(self,m1,m2):
        self.m1=m1
        self.m2=m2
    def sum(self,a,b,c):
        s=0
        if a!= None and b!= None and c!= None:
            s=a+b+c
        elif a!= None and b!= None:
            s=a+b
        else:
            s=a
        return s
s1=student(50,68)
print(s1.sum(12,85,34))
# Trick for method overloading in python but it is not method overloading
```

Out

131

In [7]

```
#Method overriding
#in start I have no phone my dad nokia phone is my phone after some time I have motorola then I tell Ihave motorola
class A:
    def show(self):
        print('in A show')
class B(A):
    pass # gives error
    def show(self):
        print('in B show')
a1=B()
a1.show()
```

Out

in B show

In [8]

```
#Abstract class and Abstract method in python
#python bydefault dont support abstract class but
# using ABC (Abstract Base Classes) module to achieve abstract classes
# What is abstract class and why it is needed
class computer:
    def process(self):# only declaration not definition then abstract method, class having abstarct method is called abstarct class
        pass
com=computer()
com.process()# we can not create object of an abstarct class
# This is not an abstract
```

In [9]

```
from abc import ABC,abstractmethod
class computer(ABC):
    @abstractmethod #decorator
    def process(self):
        pass
class Laptop:
    def process(self):# if this function is not defined laptop class is also abstarct class
        print('its running')
com=Laptop()
#abstract class have atleast one abstract method
com.process()
```

Out

its running

In [10]

```
from abc import ABC, abstractmethod
class computer(ABC):
    @abstractmethod #decorator
    def process(self):
        pass
class Laptop:
    def process(self):
        print('its running')
class whiteboard:
    def write(self):
        print('its writing')
class programmer:
    def work(self, com):
        print('solving bugs')
        com.process()

com1=Laptop()
prog1=programmer()
com2=whiteboard()
#prog1.work(com1)

prog1.work(com2)
# pass computer to whiteboard, it is compulsion to whiteboard class to have process method
# it is design aspects-design your class design your application it is not so that you cant write code without abstract class
# the other class having some signature or some restrictions to which method to define
# assume I define API and someone wants to use my API they have to define all the methods
```

Out

```
solving bugs
```

```
-----
AttributeError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_2440\1202246071.py in <module>
     20 #prog1.work(com1)
     21
--> 22 prog1.work(com2)# pass computer to whiteboard to resolve error

~\AppData\Local\Temp\ipykernel_2440\1202246071.py in work(self, com)
     13     def work(self, com):
     14         print('solving bugs')
--> 15         com.process()
     16
     17 com1=Laptop()

AttributeError: 'whiteboard' object has no attribute 'process'
```

In [11]

```
#iterator in python
nums=[7,8,9,5]
print(nums[3])
#print(nums[7])
for i in nums:
    print(i)
```

Out

```
5
7
8
9
5
```

In [12]

```
nums=[7,8,9,5]
it=iter(nums)
print(it.__next__())
print(it.__next__())
print(next(it))
for i in nums:
    print(i)
```

Out

```
7
8
9
7
8
9
5
```

In [13]

```
class Toptens:
    def __init__(self):
        self.num=1
    def __iter__(self):
        return self
    def __next__(self):
        if self.num<=10:
            val=self.num
            self.num+=1
            return val
        else:
            raise StopIteration
values =Toptens()

#print(values.__next__())
#print(values.__next__())
#own iterator
#iter will give you an object of iterator
#next will give you the next object
#only one tome 1 is printed
print(next(values))
for i in values:
    print(i)
# print and to fetch one value at a time
```

Out

```
1
2
3
4
5
6
7
8
9
10
```

In [14]

```
#Generator in python
#issues in iterator, you need to get the iterator (by defining the two function)
#python to do for us, generator gives iterator
def Toptens():
    yield 1
    yield 2
    yield 3
    yield 4
values=Toptens()
print(values.__next__())
print(values.__next__())
for i in values:
    print(i)
```

Out

```
1
2
3
4
```

In [15]

```
def Toptens():
    n=1
    while n<=10:
        sq=n*n
        yield sq
        n+=1
values=Toptens()
#for i in values:
#    print(i)
```

class3.ipynb

In [1]

```
#Exception handling in python
#compile time error=syntactical error-easiest to find (error by developer)
#logical error-wrong output- easy (bugs)(error by developer)
#run time error- wrong input is given by user(6/0)-difficult
#(error by user)#internet connection, database connection#execution should not stop
a=5
b=0
print(a/b)
print('bye')
```

Out

```
-----
ZeroDivisionError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_15296\3449676785.py in <module>
      6 a=5
      7 b=0
----> 8 print(a/b)
      9 print('bye')

ZeroDivisionError: division by zero
```

In [2]

```
a=5
b=0
try:
    print(a/b)
except Exception:
    print('Hey you can not devide a number by zero')
print('bye')
#it will jump out from except block if there is no error
```

Out

```
Hey you can not devide a number by zero
bye
```

In [3]

```
a=5
b=0
try:
    print(a/b)
except Exception as e:# e is the representation (object) of exception
    print('Hey you can not devide a number by zero',e)
print('bye')
```

Out

```
Hey you can not devide a number by zero division by zero
bye
```

In [4]

```
a=5
b=0
try:
    print('resource open')# database or file(freeze example)
    print(a/b)
    print('resource closed')
except Exception as e:#re is the representation (object) of exception
    print('Hey you can not devide a number by zero',e)
print('bye')
```

Out

```
resource open
Hey you can not devide a number by zero division by zero
bye
```

In [5]

```
a=5
b=0
#check for b=2
try:
    print('resource open')
    print(a/b)

except Exception as e:
    print('Hey you can not devide a number by zero',e)
    print('resource closed')
print('bye')
```

Out

```
resource open
Hey you can not devide a number by zero division by zero
resource closed
bye
```

In [6]

```
a=5
b=2
#check for b=2

try:
    print('resource open')
    print(a/b)
    k=int(input('Enter a number'))
    print(k)
except ZeroDivisionError as e:
    print('Hey you can not devide a number by zero',e)
except ValueError as e:
    print('Invalid number')
except Exception as e:
    print('Something went wrong')
finally:
    print('resource closed')
print('bye')
#finally block will be executed if we get error as well as we dont get error
```

Out

```
resource open
2.5
Enter a numberj
Invalid number
resource closed
bye
```

In [7]

```
#Multithreading
class Hello:
    def run(self):
        for i in range(5):
            print ('Hello')
class Hi:
    def run(self):
        for i in range(5):
            print ('Hi')
t1=Hello()
t2=Hi()
t1.run()
t2.run()
#Main thread by default
```

Out

```
Hello
Hello
Hello
Hello
Hello
Hi
Hi
Hi
Hi
Hi
Hi
```

In [8]

```
from threading import*
class Hello(Thread):
    def run(self):
        for i in range(500):
            print ('Hello')
class Hi(Thread):
    def run(self):
        for i in range(500):
            print ('Hi')
t1=Hello()
t2=Hi()
t1.start()
t2.start()
# run method in threading, other method will not work
#both threads are running parallel
```

Out

```
Hello
Hello
Hello
Hello
Hello
Hello
HiHello
Hi
Hi
Hi

Hello
Hello
Hello
Hello
Hello
```


[illegible][illegible]

Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi

[illegible]

```
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
HelloHi
```

[illegible]

[illegible]

Hi
Hi
HiHello
Hello
Hello
Hi
Hi
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello

[illegible]

[illegible]

[illegible]

[illegible][illegible]

[illegible]

```
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
HelloHi
Hi
Hi
Hi
Hi
Hi
```

```
Hello
Hello
Hello
Hello
Hello
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
Hi
```

In [9]

```
from time import sleep
from threading import*
class Hello(Thread):
    def run(self):
        for i in range(5):
            print ('Hello')
            sleep(1)
class Hi(Thread):
    def run(self):
        for i in range(5):
            print ('Hi')
            sleep(1)

t1=Hello()
t2=Hi()
t1.start()
sleep(0.5)
t2.start()
t1.join()# wait main thread to complete t1
t2.join()
print('bye')
#collision(both thread t1 and t2 comes to the cpu same time)
```

Out

```
Hello
Hi
Hello
Hi
Hello
Hi
Hello
Hi
Hello
Hi
Hi
bye
```

In [10]

```
#file handling(permanent storage)
#sql
#f=open('MyData', 'r')
#print(f.read())
f1=open('abc','w')
f1.write('Mobile')
f1=open('abc','a')
f1.write('Laptop')
f1=open('abc','r')
f1.read()
```

Out

```
'MobileLaptop'
```

In [11]

```
f=open('MyData', 'r')
f1=open('abc','w')
for data in f:
    f1.write(data)
#code for writing all the data of file mydata to abc
```

Out

```
-----
FileNotFoundError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_1576/916579558.py in <module>
----> 1 f=open('MyData', 'r')
      2 f1=open('abc','w')
      3 for data in f:
      4     f1.write(data)
      5     #code for writing all the data of file mydata to abc

FileNotFoundError: [Errno 2] No such file or directory: 'MyData'
```

In [12]

```
#image read and copy
f=open('img1', 'rb')
f1=open('img2', 'wb')
for i in f:
    f1.write(f)
```

Out

```
-----
FileNotFoundError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_1576/556252155.py in <module>
      1 #image read and copy
----> 2 f=open('img1', 'rb')
      3 f1=open('img2', 'wb')
      4 for i in f:
      5     f1.write(f)

FileNotFoundError: [Errno 2] No such file or directory: 'img1'
```

In [13]

```
#triple dots is used for documentation not comment, avoid for comments used #
```

In [14]

```
# python uses compiler or interpreter answer is both
#sourcecode-compiler-Bytecode-interpreter(python virtual machine)(pvm)-machinelanguage
#bytecode to achieve portability platform independent(different cpuarchitecture)
#cpython is implemented in c language(example of dosa)
#jython
```

data_structure.ipynb

In [1]

```
#linear search
pos=-1
def search(lst,n):
    i=0
    while i<len(lst):
        if lst[i]==n:
            globals()['pos']=i
            return True
        i=i+1
    return False
lst=[5,8,4,6,9]
n=9
if search(lst,n):
    print('Found at', pos+1)
else:
    print('not found')
```

Out

Found at 5

In [2]

```
#Binary search
#assumption-all the value should be sorted
pos=-1
def search(lst,n):
    l=0
    u=len(lst)-1
    while l <= u:
        mid =(l+u) //2
        print(mid)
        if lst[mid]==n:
            globals()['pos']=mid
            return True
        else:
            if lst[mid]<n:
                l=mid+1
            else:
                u=mid-1
    return False

lst=[4,7,8,12,45,78,47]
n=12
if search(lst,n):
    print('Found at', pos+1)
else:
    print('not found')
```

Out

3
Found at 4

In [3]

```
#Bubble_sort
def sort(nums):
    for i in range(len(nums)-1,0,-1):
        for j in range(i):
            if nums[j]>nums[j+1]:
                temp=nums[j]
                nums[j]=nums[j+1]
                nums[j+1]=temp
nums=[5,6,2,7,4,9]
sort(nums)
print(nums)
```

Out

[2, 4, 5, 6, 7, 9]

6

In [4]

```
#selection_sort
#less swapping
def sort(nums):
    for i in range(len(nums)-1):
        minpos=i
        for j in range(i,len(nums)):
            if nums[j]<nums[minpos]:
                minpos=j
        temp=nums[i]
        nums[i]=nums[minpos]
        nums[minpos]=temp
nums=[5,6,2,7,4,9]
sort(nums)
print(nums)
len(nums)
```

Out

```
[2, 4, 5, 6, 7, 9]
```

```
6
```

django.ipynb

In [1]

```
#django-free and open source web application framework
# framework is a combination of components and package- if not use framework time compusing
#website-HTML-CSS-(DESIGNING) JAVA SCRIPT  MAKES(INTRACTIVE)-front end
#backend, asp, java, php, python
# Model (data) view(page) controller(processing) (MVC) (django is faster, components are available, security, scalability)
# Model view template(MVT) view is same as template and view is same as controller
```

In [2]

```
# download python
#cmd check python version python --version, pip --version
#install django (virtual enviroment wrapper)
# virtualenvwrapper-win
#mkvirtualenv test
#pip install django
#django-admin --version
#mkdir projects
#cd projects
#django-admin startproject fahad
#python manage.py runserver(lightweight server django provides)
#type localhost address on search engine
```

In [3]

```
#static pages and dynamic pages(dynamic pages are different for each user(bank))
#download vs and install
#open the fahad project (explain each files)
#big web application is a combination of different app
#go to terminal of vs
# workon test(environment name)
#python manage.py startapp calc(app name)
#model file is used  orm basically used to hold data
#test file to test application
# view will handale the request
```

In [4]

```
#home page
```

function.ipynb

In [1]

```
def great():  
    print("hello")  
    print("function")  
great()
```

In [3]

```
def add(x,y):  
    c=x+y  
    print(c)  
add(3,4)
```

Out

```
7
```

In [4]

```
def add(x,y):  
    c=x+y  
    return c  
result=add(2,3)
```

In [5]

```
def add_sub(x,y):  
    c=x+y  
    d=x-y  
    return c,d  
results1, results2=add_sub(4,5)
```

In [6]

```
def update (x):  
    x=8  
    print(x)  
update(10)
```

Out

```
8
```

In [7]

```
def update (x):  
    print(id(x))  
    x=8  
    print(x)  
a=10  
print(id(a))  
update(a)  
print("a=", a)
```

Out

```
2076455037520  
2076455037520  
8  
a= 10
```

In [8]

```
def update (x):  
    x=8  
    print(id(x))  
    print(x)  
a=10  
print(id(a))  
update(a)  
print("a=", a)
```

Out

```
1533078366800  
1533078366736  
8  
a= 10
```

```
def update (lst): print(id(lst)) x=8 print(x) a=[1,2,2] print(id(a)) update(a) print("a=", a)
```

In [9]

```
def update (lst):  
    print(id(lst))  
    lst[1]=25  
lst=[10,20,30]  
print(id(lst))  
update(lst)  
print("lst=", lst)
```

Out

```
1533155668096  
1533155668096  
lst= [10, 25, 30]
```

In [10]

```
def add(a,b):  
    c=a+b  
    print(c)  
add(5,6)
```

Out

```
11
```

In [11]

```
def person(name,age):  
    print(name)  
    print(age-5)  
#person("fahad",30)  
#person(30,"fahad")  
person(age=30,name="fahad")
```

Out

```
fahad  
25
```

In [12]

```
def person(name,age=18):  
    print(name)  
    print(age-5)  
person("fahad")
```

Out

```
fahad  
13
```

In [13]

```
def add(a,*b):  
    c=a  
    for i in b:  
        c=c+i  
  
    print(c)  
  
add(5,6,7)
```

Out

```
18
```

In [14]

```
def person(name,**data):  
    print(name)  
    print(data)  
person("fahad", age=30, city="patna",Mob= 9998)
```

Out

```
fahad  
{'age': 30, 'city': 'patna', 'Mob': 9998}
```

In [15]

```
def person(name,**data):  
    for i,j in data.items():  
        print(i,j)  
person("fahad", age=30, city="patna",Mob= 9998)
```

Out

```
age 30  
city patna  
Mob 9998
```

In [16]

```
a=10
def something():
    global a
    a=15
    b=8
    print("in function:",a)
something()
print("out function:",a)
```

Out

```
in function: 15
out function: 15
```

In [17]

```
a=10
print(id(a))
def something():
    a=9
    x=globals() ['a']
    print(id(x))
    print("in function:",a)
    globals()['a']=15
something()
print("out function:",a)
```

Out

```
1533078366800
1533078366800
in function: 9
out function: 15
```

In [18]

```
def count(lst):
    even=0
    odd=0
    for i in lst:
        if i%2==0:
            even+=1
        else:
            odd+=1
    return even,odd

lst=[20,24,18,17,16,27]
even,odd=count(lst)
print (even)
print (odd)
print("Even:{} and Odd:{}".format(even,odd))
```

Out

```
4
2
Even:4 and Odd:2
```

In [19]

```
def fib(n):
    a=0
    b=1
    if n==1:
        print(a)
    else:
        print(a)
        print(b)
    for i in range(2,n):
        c=a+b
        a=b
        b=c
        print(c)

fib(5)
# print last number which is less than 100
```

Out

```
0
1
1
2
3
```

In [20]

```
def fact (n):  
    f=1  
    for i in range(1, n+1):  
        f=f*i  
    return f  
x=4  
result=fact(x)  
print(result)
```

Out

24

In [21]

```
import sys  
sys.setrecursionlimit(2000)  
print(sys.getrecursionlimit())  
i=0  
def great():  
    global i  
    i=i+1  
    print('hello',i)  
    great()  
  
great()
```

Out

2000
hello 1
hello 2
hello 3
hello 4
hello 5
hello 6
hello 7
hello 8
hello 9
hello 10
hello 11
hello 12
hello 13
hello 14
hello 15
hello 16
hello 17
hello 18
hello 19
hello 20
hello 21
hello 22
hello 23
hello 24
hello 25
hello 26
hello 27
hello 28
hello 29
hello 30
hello 31
hello 32
hello 33
hello 34
hello 35
hello 36
hello 37
hello 38
hello 39
hello 40
hello 41
hello 42
hello 43
hello 44
hello 45
hello 46
hello 47
hello 48
hello 49
hello 50
hello 51
hello 52
hello 53
hello 54
hello 55
hello 56
hello 57
hello 58
hello 59
hello 60
hello 61
hello 62
hello 63
hello 64
hello 65

hello 66
hello 67
hello 68
hello 69
hello 70
hello 71
hello 72
hello 73
hello 74
hello 75
hello 76
hello 77
hello 78
hello 79
hello 80
hello 81
hello 82
hello 83
hello 84
hello 85
hello 86
hello 87
hello 88
hello 89
hello 90
hello 91
hello 92
hello 93
hello 94
hello 95
hello 96
hello 97
hello 98
hello 99
hello 100
hello 101
hello 102
hello 103
hello 104
hello 105
hello 106
hello 107
hello 108
hello 109
hello 110
hello 111
hello 112
hello 113
hello 114
hello 115
hello 116
hello 117
hello 118
hello 119
hello 120
hello 121
hello 122
hello 123
hello 124
hello 125
hello 126
hello 127
hello 128
hello 129
hello 130
hello 131
hello 132
hello 133
hello 134
hello 135
hello 136
hello 137
hello 138
hello 139
hello 140
hello 141
hello 142
hello 143
hello 144
hello 145
hello 146
hello 147
hello 148
hello 149
hello 150
hello 151
hello 152
hello 153
hello 154
hello 155
hello 156
hello 157
hello 158
hello 159
hello 160

hello 161
hello 162
hello 163
hello 164
hello 165
hello 166
hello 167
hello 168
hello 169
hello 170
hello 171
hello 172
hello 173
hello 174
hello 175
hello 176
hello 177
hello 178
hello 179
hello 180
hello 181
hello 182
hello 183
hello 184
hello 185
hello 186
hello 187
hello 188
hello 189
hello 190
hello 191
hello 192
hello 193
hello 194
hello 195
hello 196
hello 197
hello 198
hello 199
hello 200
hello 201
hello 202
hello 203
hello 204
hello 205
hello 206
hello 207
hello 208
hello 209
hello 210
hello 211
hello 212
hello 213
hello 214
hello 215
hello 216
hello 217
hello 218
hello 219
hello 220
hello 221
hello 222
hello 223
hello 224
hello 225
hello 226
hello 227
hello 228
hello 229
hello 230
hello 231
hello 232
hello 233
hello 234
hello 235
hello 236
hello 237
hello 238
hello 239
hello 240
hello 241
hello 242
hello 243
hello 244
hello 245
hello 246
hello 247
hello 248
hello 249
hello 250
hello 251
hello 252
hello 253
hello 254
hello 255

hello 256
hello 257
hello 258
hello 259
hello 260
hello 261
hello 262
hello 263
hello 264
hello 265
hello 266
hello 267
hello 268
hello 269
hello 270
hello 271
hello 272
hello 273
hello 274
hello 275
hello 276
hello 277
hello 278
hello 279
hello 280
hello 281
hello 282
hello 283
hello 284
hello 285
hello 286
hello 287
hello 288
hello 289
hello 290
hello 291
hello 292
hello 293
hello 294
hello 295
hello 296
hello 297
hello 298
hello 299
hello 300
hello 301
hello 302
hello 303
hello 304
hello 305
hello 306
hello 307
hello 308
hello 309
hello 310
hello 311
hello 312
hello 313
hello 314
hello 315
hello 316
hello 317
hello 318
hello 319
hello 320
hello 321
hello 322
hello 323
hello 324
hello 325
hello 326
hello 327
hello 328
hello 329
hello 330
hello 331
hello 332
hello 333
hello 334
hello 335
hello 336
hello 337
hello 338
hello 339
hello 340
hello 341
hello 342
hello 343
hello 344
hello 345
hello 346
hello 347
hello 348
hello 349
hello 350

hello 351
hello 352
hello 353
hello 354
hello 355
hello 356
hello 357
hello 358
hello 359
hello 360
hello 361
hello 362
hello 363
hello 364
hello 365
hello 366
hello 367
hello 368
hello 369
hello 370
hello 371
hello 372
hello 373
hello 374
hello 375
hello 376
hello 377
hello 378
hello 379
hello 380
hello 381
hello 382
hello 383
hello 384
hello 385
hello 386
hello 387
hello 388
hello 389
hello 390
hello 391
hello 392
hello 393
hello 394
hello 395
hello 396
hello 397
hello 398
hello 399
hello 400
hello 401
hello 402
hello 403
hello 404
hello 405
hello 406
hello 407
hello 408
hello 409
hello 410
hello 411
hello 412
hello 413
hello 414
hello 415
hello 416
hello 417
hello 418
hello 419
hello 420
hello 421
hello 422
hello 423
hello 424
hello 425
hello 426
hello 427
hello 428
hello 429
hello 430
hello 431
hello 432
hello 433
hello 434
hello 435
hello 436
hello 437
hello 438
hello 439
hello 440
hello 441
hello 442
hello 443
hello 444
hello 445

hello 446
hello 447
hello 448
hello 449
hello 450
hello 451
hello 452
hello 453
hello 454
hello 455
hello 456
hello 457
hello 458
hello 459
hello 460
hello 461
hello 462
hello 463
hello 464
hello 465
hello 466
hello 467
hello 468
hello 469
hello 470
hello 471
hello 472
hello 473
hello 474
hello 475
hello 476
hello 477
hello 478
hello 479
hello 480
hello 481
hello 482
hello 483
hello 484
hello 485
hello 486
hello 487
hello 488
hello 489
hello 490
hello 491
hello 492
hello 493
hello 494
hello 495
hello 496
hello 497
hello 498
hello 499
hello 500
hello 501
hello 502
hello 503
hello 504
hello 505
hello 506
hello 507
hello 508
hello 509
hello 510
hello 511
hello 512
hello 513
hello 514
hello 515
hello 516
hello 517
hello 518
hello 519
hello 520
hello 521
hello 522
hello 523
hello 524
hello 525
hello 526
hello 527
hello 528
hello 529
hello 530
hello 531
hello 532
hello 533
hello 534
hello 535
hello 536
hello 537
hello 538
hello 539
hello 540

hello 541
hello 542
hello 543
hello 544
hello 545
hello 546
hello 547
hello 548
hello 549
hello 550
hello 551
hello 552
hello 553
hello 554
hello 555
hello 556
hello 557
hello 558
hello 559
hello 560
hello 561
hello 562
hello 563
hello 564
hello 565
hello 566
hello 567
hello 568
hello 569
hello 570
hello 571
hello 572
hello 573
hello 574
hello 575
hello 576
hello 577
hello 578
hello 579
hello 580
hello 581
hello 582
hello 583
hello 584
hello 585
hello 586
hello 587
hello 588
hello 589
hello 590
hello 591
hello 592
hello 593
hello 594
hello 595
hello 596
hello 597
hello 598
hello 599
hello 600
hello 601
hello 602
hello 603
hello 604
hello 605
hello 606
hello 607
hello 608
hello 609
hello 610
hello 611
hello 612
hello 613
hello 614
hello 615
hello 616
hello 617
hello 618
hello 619
hello 620
hello 621
hello 622
hello 623
hello 624
hello 625
hello 626
hello 627
hello 628
hello 629
hello 630
hello 631
hello 632
hello 633
hello 634
hello 635

hello 636
hello 637
hello 638
hello 639
hello 640
hello 641
hello 642
hello 643
hello 644
hello 645
hello 646
hello 647
hello 648
hello 649
hello 650
hello 651
hello 652
hello 653
hello 654
hello 655
hello 656
hello 657
hello 658
hello 659
hello 660
hello 661
hello 662
hello 663
hello 664
hello 665
hello 666
hello 667
hello 668
hello 669
hello 670
hello 671
hello 672
hello 673
hello 674
hello 675
hello 676
hello 677
hello 678
hello 679
hello 680
hello 681
hello 682
hello 683
hello 684
hello 685
hello 686
hello 687
hello 688
hello 689
hello 690
hello 691
hello 692
hello 693
hello 694
hello 695
hello 696
hello 697
hello 698
hello 699
hello 700
hello 701
hello 702
hello 703
hello 704
hello 705
hello 706
hello 707
hello 708
hello 709
hello 710
hello 711
hello 712
hello 713
hello 714
hello 715
hello 716
hello 717
hello 718
hello 719
hello 720
hello 721
hello 722
hello 723
hello 724
hello 725
hello 726
hello 727
hello 728
hello 729
hello 730

hello 731
hello 732
hello 733
hello 734
hello 735
hello 736
hello 737
hello 738
hello 739
hello 740
hello 741
hello 742
hello 743
hello 744
hello 745
hello 746
hello 747
hello 748
hello 749
hello 750
hello 751
hello 752
hello 753
hello 754
hello 755
hello 756
hello 757
hello 758
hello 759
hello 760
hello 761
hello 762
hello 763
hello 764
hello 765
hello 766
hello 767
hello 768
hello 769
hello 770
hello 771
hello 772
hello 773
hello 774
hello 775
hello 776
hello 777
hello 778
hello 779
hello 780
hello 781
hello 782
hello 783
hello 784
hello 785
hello 786
hello 787
hello 788
hello 789
hello 790
hello 791
hello 792
hello 793
hello 794
hello 795
hello 796
hello 797
hello 798
hello 799
hello 800
hello 801
hello 802
hello 803
hello 804
hello 805
hello 806
hello 807
hello 808
hello 809
hello 810
hello 811
hello 812
hello 813
hello 814
hello 815
hello 816
hello 817
hello 818
hello 819
hello 820
hello 821
hello 822
hello 823
hello 824
hello 825

hello 826
hello 827
hello 828
hello 829
hello 830
hello 831
hello 832
hello 833
hello 834
hello 835
hello 836
hello 837
hello 838
hello 839
hello 840
hello 841
hello 842
hello 843
hello 844
hello 845
hello 846
hello 847
hello 848
hello 849
hello 850
hello 851
hello 852
hello 853
hello 854
hello 855
hello 856
hello 857
hello 858
hello 859
hello 860
hello 861
hello 862
hello 863
hello 864
hello 865
hello 866
hello 867
hello 868
hello 869
hello 870
hello 871
hello 872
hello 873
hello 874
hello 875
hello 876
hello 877
hello 878
hello 879
hello 880
hello 881
hello 882
hello 883
hello 884
hello 885
hello 886
hello 887
hello 888
hello 889
hello 890
hello 891
hello 892
hello 893
hello 894
hello 895
hello 896
hello 897
hello 898
hello 899
hello 900
hello 901
hello 902
hello 903
hello 904
hello 905
hello 906
hello 907
hello 908
hello 909
hello 910
hello 911
hello 912
hello 913
hello 914
hello 915
hello 916
hello 917
hello 918
hello 919
hello 920

hello 921
hello 922
hello 923
hello 924
hello 925
hello 926
hello 927
hello 928
hello 929
hello 930
hello 931
hello 932
hello 933
hello 934
hello 935
hello 936
hello 937
hello 938
hello 939
hello 940
hello 941
hello 942
hello 943
hello 944
hello 945
hello 946
hello 947
hello 948
hello 949
hello 950
hello 951
hello 952
hello 953
hello 954
hello 955
hello 956
hello 957
hello 958
hello 959
hello 960
hello 961
hello 962
hello 963
hello 964
hello 965
hello 966
hello 967
hello 968
hello 969
hello 970
hello 971
hello 972
hello 973
hello 974
hello 975
hello 976
hello 977
hello 978
hello 979
hello 980
hello 981
hello 982
hello 983
hello 984
hello 985
hello 986
hello 987
hello 988
hello 989
hello 990
hello 991
hello 992
hello 993
hello 994
hello 995
hello 996
hello 997
hello 998
hello 999
hello 1000
hello 1001
hello 1002
hello 1003
hello 1004
hello 1005
hello 1006
hello 1007
hello 1008
hello 1009
hello 1010
hello 1011
hello 1012
hello 1013
hello 1014
hello 1015

hello 1016
hello 1017
hello 1018
hello 1019
hello 1020
hello 1021
hello 1022
hello 1023
hello 1024
hello 1025
hello 1026
hello 1027
hello 1028
hello 1029
hello 1030
hello 1031
hello 1032
hello 1033
hello 1034
hello 1035
hello 1036
hello 1037
hello 1038
hello 1039
hello 1040
hello 1041
hello 1042
hello 1043
hello 1044
hello 1045
hello 1046
hello 1047
hello 1048
hello 1049
hello 1050
hello 1051
hello 1052
hello 1053
hello 1054
hello 1055
hello 1056
hello 1057
hello 1058
hello 1059
hello 1060
hello 1061
hello 1062
hello 1063
hello 1064
hello 1065
hello 1066
hello 1067
hello 1068
hello 1069
hello 1070
hello 1071
hello 1072
hello 1073
hello 1074
hello 1075
hello 1076
hello 1077
hello 1078
hello 1079
hello 1080
hello 1081
hello 1082
hello 1083
hello 1084
hello 1085
hello 1086
hello 1087
hello 1088
hello 1089
hello 1090
hello 1091
hello 1092
hello 1093
hello 1094
hello 1095
hello 1096
hello 1097
hello 1098
hello 1099
hello 1100
hello 1101
hello 1102
hello 1103
hello 1104
hello 1105
hello 1106
hello 1107
hello 1108
hello 1109
hello 1110

hello 1111
hello 1112
hello 1113
hello 1114
hello 1115
hello 1116
hello 1117
hello 1118
hello 1119
hello 1120
hello 1121
hello 1122
hello 1123
hello 1124
hello 1125
hello 1126
hello 1127
hello 1128
hello 1129
hello 1130
hello 1131
hello 1132
hello 1133
hello 1134
hello 1135
hello 1136
hello 1137
hello 1138
hello 1139
hello 1140
hello 1141
hello 1142
hello 1143
hello 1144
hello 1145
hello 1146
hello 1147
hello 1148
hello 1149
hello 1150
hello 1151
hello 1152
hello 1153
hello 1154
hello 1155
hello 1156
hello 1157
hello 1158
hello 1159
hello 1160
hello 1161
hello 1162
hello 1163
hello 1164
hello 1165
hello 1166
hello 1167
hello 1168
hello 1169
hello 1170
hello 1171
hello 1172
hello 1173
hello 1174
hello 1175
hello 1176
hello 1177
hello 1178
hello 1179
hello 1180
hello 1181
hello 1182
hello 1183
hello 1184
hello 1185
hello 1186
hello 1187
hello 1188
hello 1189
hello 1190
hello 1191
hello 1192
hello 1193
hello 1194
hello 1195
hello 1196
hello 1197
hello 1198
hello 1199
hello 1200
hello 1201
hello 1202
hello 1203
hello 1204
hello 1205

hello 1206
hello 1207
hello 1208
hello 1209
hello 1210
hello 1211
hello 1212
hello 1213
hello 1214
hello 1215
hello 1216
hello 1217
hello 1218
hello 1219
hello 1220
hello 1221
hello 1222
hello 1223
hello 1224
hello 1225
hello 1226
hello 1227
hello 1228
hello 1229
hello 1230
hello 1231
hello 1232
hello 1233
hello 1234
hello 1235
hello 1236
hello 1237
hello 1238
hello 1239
hello 1240
hello 1241
hello 1242
hello 1243
hello 1244
hello 1245
hello 1246
hello 1247
hello 1248
hello 1249
hello 1250
hello 1251
hello 1252
hello 1253
hello 1254
hello 1255
hello 1256
hello 1257
hello 1258
hello 1259
hello 1260
hello 1261
hello 1262
hello 1263
hello 1264
hello 1265
hello 1266
hello 1267
hello 1268
hello 1269
hello 1270
hello 1271
hello 1272
hello 1273
hello 1274
hello 1275
hello 1276
hello 1277
hello 1278
hello 1279
hello 1280
hello 1281
hello 1282
hello 1283
hello 1284
hello 1285
hello 1286
hello 1287
hello 1288
hello 1289
hello 1290
hello 1291
hello 1292
hello 1293
hello 1294
hello 1295
hello 1296
hello 1297
hello 1298
hello 1299
hello 1300

hello 1301
hello 1302
hello 1303
hello 1304
hello 1305
hello 1306
hello 1307
hello 1308
hello 1309
hello 1310
hello 1311
hello 1312
hello 1313
hello 1314
hello 1315
hello 1316
hello 1317
hello 1318
hello 1319
hello 1320
hello 1321
hello 1322
hello 1323
hello 1324
hello 1325
hello 1326
hello 1327
hello 1328
hello 1329
hello 1330
hello 1331
hello 1332
hello 1333
hello 1334
hello 1335
hello 1336
hello 1337
hello 1338
hello 1339
hello 1340
hello 1341
hello 1342
hello 1343
hello 1344
hello 1345
hello 1346
hello 1347
hello 1348
hello 1349
hello 1350
hello 1351
hello 1352
hello 1353
hello 1354
hello 1355
hello 1356
hello 1357
hello 1358
hello 1359
hello 1360
hello 1361
hello 1362
hello 1363
hello 1364
hello 1365
hello 1366
hello 1367
hello 1368
hello 1369
hello 1370
hello 1371
hello 1372
hello 1373
hello 1374
hello 1375
hello 1376
hello 1377
hello 1378
hello 1379
hello 1380
hello 1381
hello 1382
hello 1383
hello 1384
hello 1385
hello 1386
hello 1387
hello 1388
hello 1389
hello 1390
hello 1391
hello 1392
hello 1393
hello 1394
hello 1395

hello 1396
hello 1397
hello 1398
hello 1399
hello 1400
hello 1401
hello 1402
hello 1403
hello 1404
hello 1405
hello 1406
hello 1407
hello 1408
hello 1409
hello 1410
hello 1411
hello 1412
hello 1413
hello 1414
hello 1415
hello 1416
hello 1417
hello 1418
hello 1419
hello 1420
hello 1421
hello 1422
hello 1423
hello 1424
hello 1425
hello 1426
hello 1427
hello 1428
hello 1429
hello 1430
hello 1431
hello 1432
hello 1433
hello 1434
hello 1435
hello 1436
hello 1437
hello 1438
hello 1439
hello 1440
hello 1441
hello 1442
hello 1443
hello 1444
hello 1445
hello 1446
hello 1447
hello 1448
hello 1449
hello 1450
hello 1451
hello 1452
hello 1453
hello 1454
hello 1455
hello 1456
hello 1457
hello 1458
hello 1459
hello 1460
hello 1461
hello 1462
hello 1463
hello 1464
hello 1465
hello 1466
hello 1467
hello 1468
hello 1469
hello 1470
hello 1471
hello 1472
hello 1473
hello 1474
hello 1475
hello 1476
hello 1477
hello 1478
hello 1479
hello 1480
hello 1481
hello 1482
hello 1483
hello 1484
hello 1485
hello 1486
hello 1487
hello 1488
hello 1489
hello 1490

hello 1491
hello 1492
hello 1493
hello 1494
hello 1495
hello 1496
hello 1497
hello 1498
hello 1499
hello 1500
hello 1501
hello 1502
hello 1503
hello 1504
hello 1505
hello 1506
hello 1507
hello 1508
hello 1509
hello 1510
hello 1511
hello 1512
hello 1513
hello 1514
hello 1515
hello 1516
hello 1517
hello 1518
hello 1519
hello 1520
hello 1521
hello 1522
hello 1523
hello 1524
hello 1525
hello 1526
hello 1527
hello 1528
hello 1529
hello 1530
hello 1531
hello 1532
hello 1533
hello 1534
hello 1535
hello 1536
hello 1537
hello 1538
hello 1539
hello 1540
hello 1541
hello 1542
hello 1543
hello 1544
hello 1545
hello 1546
hello 1547
hello 1548
hello 1549
hello 1550
hello 1551
hello 1552
hello 1553
hello 1554
hello 1555
hello 1556
hello 1557
hello 1558
hello 1559
hello 1560
hello 1561
hello 1562
hello 1563
hello 1564
hello 1565
hello 1566
hello 1567
hello 1568
hello 1569
hello 1570
hello 1571
hello 1572
hello 1573
hello 1574
hello 1575
hello 1576
hello 1577
hello 1578
hello 1579
hello 1580
hello 1581
hello 1582
hello 1583
hello 1584
hello 1585

hello 1586
hello 1587
hello 1588
hello 1589
hello 1590
hello 1591
hello 1592
hello 1593
hello 1594
hello 1595
hello 1596
hello 1597
hello 1598
hello 1599
hello 1600
hello 1601
hello 1602
hello 1603
hello 1604
hello 1605
hello 1606
hello 1607
hello 1608
hello 1609
hello 1610
hello 1611
hello 1612
hello 1613
hello 1614
hello 1615
hello 1616
hello 1617
hello 1618
hello 1619
hello 1620
hello 1621
hello 1622
hello 1623
hello 1624
hello 1625
hello 1626
hello 1627
hello 1628
hello 1629
hello 1630
hello 1631
hello 1632
hello 1633
hello 1634
hello 1635
hello 1636
hello 1637
hello 1638
hello 1639
hello 1640
hello 1641
hello 1642
hello 1643
hello 1644
hello 1645
hello 1646
hello 1647
hello 1648
hello 1649
hello 1650
hello 1651
hello 1652
hello 1653
hello 1654
hello 1655
hello 1656
hello 1657
hello 1658
hello 1659
hello 1660
hello 1661
hello 1662
hello 1663
hello 1664
hello 1665
hello 1666
hello 1667
hello 1668
hello 1669
hello 1670
hello 1671
hello 1672
hello 1673
hello 1674
hello 1675
hello 1676
hello 1677
hello 1678
hello 1679
hello 1680

hello 1681
hello 1682
hello 1683
hello 1684
hello 1685
hello 1686
hello 1687
hello 1688
hello 1689
hello 1690
hello 1691
hello 1692
hello 1693
hello 1694
hello 1695
hello 1696
hello 1697
hello 1698
hello 1699
hello 1700
hello 1701
hello 1702
hello 1703
hello 1704
hello 1705
hello 1706
hello 1707
hello 1708
hello 1709
hello 1710
hello 1711
hello 1712
hello 1713
hello 1714
hello 1715
hello 1716
hello 1717
hello 1718
hello 1719
hello 1720
hello 1721
hello 1722
hello 1723
hello 1724
hello 1725
hello 1726
hello 1727
hello 1728
hello 1729
hello 1730
hello 1731
hello 1732
hello 1733
hello 1734
hello 1735
hello 1736
hello 1737
hello 1738
hello 1739
hello 1740
hello 1741
hello 1742
hello 1743
hello 1744
hello 1745
hello 1746
hello 1747
hello 1748
hello 1749
hello 1750
hello 1751
hello 1752
hello 1753
hello 1754
hello 1755
hello 1756
hello 1757
hello 1758
hello 1759
hello 1760
hello 1761
hello 1762
hello 1763
hello 1764
hello 1765
hello 1766
hello 1767
hello 1768
hello 1769
hello 1770
hello 1771
hello 1772
hello 1773
hello 1774
hello 1775

hello 1776
hello 1777
hello 1778
hello 1779
hello 1780
hello 1781
hello 1782
hello 1783
hello 1784
hello 1785
hello 1786
hello 1787
hello 1788
hello 1789
hello 1790
hello 1791
hello 1792
hello 1793
hello 1794
hello 1795
hello 1796
hello 1797
hello 1798
hello 1799
hello 1800
hello 1801
hello 1802
hello 1803
hello 1804
hello 1805
hello 1806
hello 1807
hello 1808
hello 1809
hello 1810
hello 1811
hello 1812
hello 1813
hello 1814
hello 1815
hello 1816
hello 1817
hello 1818
hello 1819
hello 1820
hello 1821
hello 1822
hello 1823
hello 1824
hello 1825
hello 1826
hello 1827
hello 1828
hello 1829
hello 1830
hello 1831
hello 1832
hello 1833
hello 1834
hello 1835
hello 1836
hello 1837
hello 1838
hello 1839
hello 1840
hello 1841
hello 1842
hello 1843
hello 1844
hello 1845
hello 1846
hello 1847
hello 1848
hello 1849
hello 1850
hello 1851
hello 1852
hello 1853
hello 1854
hello 1855
hello 1856
hello 1857
hello 1858
hello 1859
hello 1860
hello 1861
hello 1862
hello 1863
hello 1864
hello 1865
hello 1866
hello 1867
hello 1868
hello 1869
hello 1870

hello 1871
hello 1872
hello 1873
hello 1874
hello 1875
hello 1876
hello 1877
hello 1878
hello 1879
hello 1880
hello 1881
hello 1882
hello 1883
hello 1884
hello 1885
hello 1886
hello 1887
hello 1888
hello 1889
hello 1890
hello 1891
hello 1892
hello 1893
hello 1894
hello 1895
hello 1896
hello 1897
hello 1898
hello 1899
hello 1900
hello 1901
hello 1902
hello 1903
hello 1904
hello 1905
hello 1906
hello 1907
hello 1908
hello 1909
hello 1910
hello 1911
hello 1912
hello 1913
hello 1914
hello 1915
hello 1916
hello 1917
hello 1918
hello 1919
hello 1920
hello 1921
hello 1922
hello 1923
hello 1924
hello 1925
hello 1926
hello 1927
hello 1928
hello 1929
hello 1930
hello 1931
hello 1932
hello 1933
hello 1934
hello 1935
hello 1936
hello 1937
hello 1938
hello 1939
hello 1940
hello 1941
hello 1942
hello 1943
hello 1944
hello 1945
hello 1946
hello 1947
hello 1948
hello 1949
hello 1950
hello 1951
hello 1952
hello 1953
hello 1954
hello 1955
hello 1956
hello 1957
hello 1958
hello 1959
hello 1960
hello 1961
hello 1962
hello 1963
hello 1964
hello 1965

```
hello 1966
hello 1967
```

```
-----
RecursionError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_17752\1763571193.py in <module>
      9     great()
     10
--> 11 great()
     12

~\AppData\Local\Temp\ipykernel_17752\1763571193.py in great()
      7     i=i+1
      8     print('hello',i)
----> 9     great()
     10
     11 great()

... last 1 frames repeated, from the frame below ...

~\AppData\Local\Temp\ipykernel_17752\1763571193.py in great()
      7     i=i+1
      8     print('hello',i)
----> 9     great()
     10
     11 great()

RecursionError: maximum recursion depth exceeded while calling a Python object
```

In [22]

```
def fact(n):
    if n==0:
        return 1
    return n*fact(n-1)
result=fact(5)
print(result)
```

Out

```
120
```

In [23]

```
#Anonymus function(function without name)
f1= lambda a: a*a
result1= f1(5)
print(result1)
f2= lambda a,b: a+b
result2= f2(5,9)
print(result2)
```

Out

```
25
14
```

In [24]

```
#def is_even(n):
#    return n%2==0
from functools import reduce
num=[2,4,8,5,7,43]
evens=list(filter(lambda n: n%2==0, num))
print(evens)
doubles=list(map(lambda n: n*2, num))
print(doubles)
red=reduce(lambda a,b: a+b , doubles)
print(red)
```

Out

```
[2, 4, 8]
[4, 8, 16, 10, 14, 86]
138
```

In [25]

```
#decorator
def div(a,b):
    print(a/b)
def smart_div(func):
    def inner(a,b):
        if a<b:
            a,b=b,a
        return (func(a,b))
    return inner
div =smart_div(div)
div(2,4)
```

Out

```
2.0
```

In [26]

```
#Modules
def add (a,b):
    return a+b
def sub (a,b):
    return a-b
def Mul (a,b):
    return a*b
def div (a,b):
    return a/b
#save as calc.py
#create new file demo.py
import calc
a=9
b=6
calc.add(a,b)
from calc import*
add(a,b)
```

In [27]

```
#demo file
#special varoable name
#print(__name__)
#calc file
print ('hello'+ __name__)
#import calc in demo
print ('demosays'+ __name__)
#demo file print name of the module
```

```
#demo file
print("hello")
print ("welcome user")
#calc file
print('its time to calculate')
#it print all
#demo file
def main():
    print("hello")
    print ("welcome user")
if __name__ == "__main__":
    main()
```

Out

```
hello__main__
demosays__main__
hello
welcome user
its time to calculate
hello
welcome user
```

function_practice.ipynb

In [1]

```
def greet():  
    print("hello")  
    print("function")
```

In [2]

```
greet()
```

Out

```
hello  
function
```

In [3]

```
def add(x,y):  
    c=x+y  
    print(c)
```

In [4]

```
add(3,4)
```

Out

```
7
```

In [5]

```
def add(x,y):  
    c=x+y  
    return c  
result=add(2,3)  
print(result)
```

Out

```
5
```

In [6]

```
def add_sub(x,y):  
    c=x+y  
    d=x-y  
    return c,d  
results1, results2=add_sub(4,5)  
print(results1)  
print(results2)
```

Out

```
9  
-1
```

In [7]

```
def update (x):  
    x=8  
    print(x)  
  
update(10)
```

Out

```
8
```

In [8]

```
def update (x):  
    print(id(x))  
    x=8  
    print(x)  
a=10  
print(id(a))  
update(a)  
print("a=", a)
```

Out

```
2188986706512  
2188986706512  
8  
a= 10
```

In [9]

```
def update (x):
    x=8
    print(id(x))
    print(x)
a=10
print(id(a))
update(a)
print("a=", a)
```

Out

```
2188986706512
2188986706448
8
a= 10
```

In [10]

```
def update (lst):
    print(id(lst))
    lst[1]=25
lst=[10,20,30]
print(id(lst))
update(lst)
print("lst=", lst)
```

Out

```
2189063836800
2189063836800
lst= [10, 25, 30]
```

In [11]

```
def person(name,age):
    print(name)
    #print(age)
    print(age-5)
#person("fahad",30)
#person(30,"fahad")
person(age=30,name="fahad")
```

Out

```
fahad
25
```

In [12]

```
def add(a,b):
    c=a+b
    print(c)
add(2,3,3)
```

Out

```
-----
TypeError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_25768\3901945440.py in <module>
      2     c=a+b
      3     print(c)
----> 4 add(2,3,3)

TypeError: add() takes 2 positional arguments but 3 were given
```

In [13]

```
def add(a,*b):
    c=a
    for i in b:
        c=c+i

    print(c)

add(5,6,7,8,9)
```

Out

```
35
```

In [14]

```
def person(name,**data):
    print(name)
    print(data)
person('fahad',age= 30,Mob=9998)
```

Out

```
fahad
{'age': 30, 'Mob': 9998}
```

In [15]

```
def person(name,**data):
    for i,j in data.items():
        print(i,j)
person("fahad", age=30, city="patna",Mob= 9998)
```

Out

```
age 30
city patna
Mob 9998
```

In [16]

```
a=10
def something():
    global a
    global b
    a=15
    b=8
    print("in function:",a)
something()
print("out function:",a)
print(b)
```

Out

```
in function: 15
out function: 15
8
```

In [17]

```
a=10
print(id(a))
def something():
    a=9
    x=globals()['a']
    print(id(x))
    print("in function:",a)
    globals()['a']=15
something()
print("out function:",a)
```

Out

```
2188986706512
2188986706512
in function: 9
out function: 15
```

In [18]

```
def count(lst):
    even=0
    odd=0
    for i in lst:
        if i%2==0:
            even+=1
        else:
            odd+=1
    return even,odd

lst=[20,24,18,17,16,27]
even1,odd1=count(lst)
print (even1)
print (odd1)
print("Even:{} and Odd:{}".format(even1,odd1))
```

Out

```
4
2
Even:4 and Odd:2
```

In [19]

```
def fib(n):
    a=0
    b=1
    if n==1:
        print(a)
    else:
        print(a)
        print(b)
    for i in range(2,n):
        c=a+b
        a=b
        b=c
        print(c)

fib(5)
# print last number which is less than 100
```

Out

```
0
1
1
2
3
```

In [20]

```
def fact (n):
    f=1
    for i in range(1, n+1):
        f=f*i
    return f

x=4
result=fact(x)
print(result)
```

Out

```
24
```

In [21]

```
import sys
sys.setrecursionlimit(2000)
print(sys.getrecursionlimit())
i=0
def great():
    global i
    i=i+1
    print('hello',i)
    great()

great()
```

Out

```
2000
hello 1
hello 2
hello 3
hello 4
hello 5
hello 6
hello 7
hello 8
hello 9
hello 10
hello 11
hello 12
hello 13
hello 14
hello 15
hello 16
hello 17
hello 18
hello 19
hello 20
hello 21
hello 22
hello 23
hello 24
hello 25
hello 26
hello 27
hello 28
hello 29
hello 30
hello 31
hello 32
hello 33
hello 34
hello 35
hello 36
hello 37
hello 38
```

hello 39
hello 40
hello 41
hello 42
hello 43
hello 44
hello 45
hello 46
hello 47
hello 48
hello 49
hello 50
hello 51
hello 52
hello 53
hello 54
hello 55
hello 56
hello 57
hello 58
hello 59
hello 60
hello 61
hello 62
hello 63
hello 64
hello 65
hello 66
hello 67
hello 68
hello 69
hello 70
hello 71
hello 72
hello 73
hello 74
hello 75
hello 76
hello 77
hello 78
hello 79
hello 80
hello 81
hello 82
hello 83
hello 84
hello 85
hello 86
hello 87
hello 88
hello 89
hello 90
hello 91
hello 92
hello 93
hello 94
hello 95
hello 96
hello 97
hello 98
hello 99
hello 100
hello 101
hello 102
hello 103
hello 104
hello 105
hello 106
hello 107
hello 108
hello 109
hello 110
hello 111
hello 112
hello 113
hello 114
hello 115
hello 116
hello 117
hello 118
hello 119
hello 120
hello 121
hello 122
hello 123
hello 124
hello 125
hello 126
hello 127
hello 128
hello 129
hello 130
hello 131
hello 132
hello 133

hello 134
hello 135
hello 136
hello 137
hello 138
hello 139
hello 140
hello 141
hello 142
hello 143
hello 144
hello 145
hello 146
hello 147
hello 148
hello 149
hello 150
hello 151
hello 152
hello 153
hello 154
hello 155
hello 156
hello 157
hello 158
hello 159
hello 160
hello 161
hello 162
hello 163
hello 164
hello 165
hello 166
hello 167
hello 168
hello 169
hello 170
hello 171
hello 172
hello 173
hello 174
hello 175
hello 176
hello 177
hello 178
hello 179
hello 180
hello 181
hello 182
hello 183
hello 184
hello 185
hello 186
hello 187
hello 188
hello 189
hello 190
hello 191
hello 192
hello 193
hello 194
hello 195
hello 196
hello 197
hello 198
hello 199
hello 200
hello 201
hello 202
hello 203
hello 204
hello 205
hello 206
hello 207
hello 208
hello 209
hello 210
hello 211
hello 212
hello 213
hello 214
hello 215
hello 216
hello 217
hello 218
hello 219
hello 220
hello 221
hello 222
hello 223
hello 224
hello 225
hello 226
hello 227
hello 228

hello 229
hello 230
hello 231
hello 232
hello 233
hello 234
hello 235
hello 236
hello 237
hello 238
hello 239
hello 240
hello 241
hello 242
hello 243
hello 244
hello 245
hello 246
hello 247
hello 248
hello 249
hello 250
hello 251
hello 252
hello 253
hello 254
hello 255
hello 256
hello 257
hello 258
hello 259
hello 260
hello 261
hello 262
hello 263
hello 264
hello 265
hello 266
hello 267
hello 268
hello 269
hello 270
hello 271
hello 272
hello 273
hello 274
hello 275
hello 276
hello 277
hello 278
hello 279
hello 280
hello 281
hello 282
hello 283
hello 284
hello 285
hello 286
hello 287
hello 288
hello 289
hello 290
hello 291
hello 292
hello 293
hello 294
hello 295
hello 296
hello 297
hello 298
hello 299
hello 300
hello 301
hello 302
hello 303
hello 304
hello 305
hello 306
hello 307
hello 308
hello 309
hello 310
hello 311
hello 312
hello 313
hello 314
hello 315
hello 316
hello 317
hello 318
hello 319
hello 320
hello 321
hello 322
hello 323

hello 324
hello 325
hello 326
hello 327
hello 328
hello 329
hello 330
hello 331
hello 332
hello 333
hello 334
hello 335
hello 336
hello 337
hello 338
hello 339
hello 340
hello 341
hello 342
hello 343
hello 344
hello 345
hello 346
hello 347
hello 348
hello 349
hello 350
hello 351
hello 352
hello 353
hello 354
hello 355
hello 356
hello 357
hello 358
hello 359
hello 360
hello 361
hello 362
hello 363
hello 364
hello 365
hello 366
hello 367
hello 368
hello 369
hello 370
hello 371
hello 372
hello 373
hello 374
hello 375
hello 376
hello 377
hello 378
hello 379
hello 380
hello 381
hello 382
hello 383
hello 384
hello 385
hello 386
hello 387
hello 388
hello 389
hello 390
hello 391
hello 392
hello 393
hello 394
hello 395
hello 396
hello 397
hello 398
hello 399
hello 400
hello 401
hello 402
hello 403
hello 404
hello 405
hello 406
hello 407
hello 408
hello 409
hello 410
hello 411
hello 412
hello 413
hello 414
hello 415
hello 416
hello 417
hello 418

hello 419
hello 420
hello 421
hello 422
hello 423
hello 424
hello 425
hello 426
hello 427
hello 428
hello 429
hello 430
hello 431
hello 432
hello 433
hello 434
hello 435
hello 436
hello 437
hello 438
hello 439
hello 440
hello 441
hello 442
hello 443
hello 444
hello 445
hello 446
hello 447
hello 448
hello 449
hello 450
hello 451
hello 452
hello 453
hello 454
hello 455
hello 456
hello 457
hello 458
hello 459
hello 460
hello 461
hello 462
hello 463
hello 464
hello 465
hello 466
hello 467
hello 468
hello 469
hello 470
hello 471
hello 472
hello 473
hello 474
hello 475
hello 476
hello 477
hello 478
hello 479
hello 480
hello 481
hello 482
hello 483
hello 484
hello 485
hello 486
hello 487
hello 488
hello 489
hello 490
hello 491
hello 492
hello 493
hello 494
hello 495
hello 496
hello 497
hello 498
hello 499
hello 500
hello 501
hello 502
hello 503
hello 504
hello 505
hello 506
hello 507
hello 508
hello 509
hello 510
hello 511
hello 512
hello 513

hello 514
hello 515
hello 516
hello 517
hello 518
hello 519
hello 520
hello 521
hello 522
hello 523
hello 524
hello 525
hello 526
hello 527
hello 528
hello 529
hello 530
hello 531
hello 532
hello 533
hello 534
hello 535
hello 536
hello 537
hello 538
hello 539
hello 540
hello 541
hello 542
hello 543
hello 544
hello 545
hello 546
hello 547
hello 548
hello 549
hello 550
hello 551
hello 552
hello 553
hello 554
hello 555
hello 556
hello 557
hello 558
hello 559
hello 560
hello 561
hello 562
hello 563
hello 564
hello 565
hello 566
hello 567
hello 568
hello 569
hello 570
hello 571
hello 572
hello 573
hello 574
hello 575
hello 576
hello 577
hello 578
hello 579
hello 580
hello 581
hello 582
hello 583
hello 584
hello 585
hello 586
hello 587
hello 588
hello 589
hello 590
hello 591
hello 592
hello 593
hello 594
hello 595
hello 596
hello 597
hello 598
hello 599
hello 600
hello 601
hello 602
hello 603
hello 604
hello 605
hello 606
hello 607
hello 608

hello 609
hello 610
hello 611
hello 612
hello 613
hello 614
hello 615
hello 616
hello 617
hello 618
hello 619
hello 620
hello 621
hello 622
hello 623
hello 624
hello 625
hello 626
hello 627
hello 628
hello 629
hello 630
hello 631
hello 632
hello 633
hello 634
hello 635
hello 636
hello 637
hello 638
hello 639
hello 640
hello 641
hello 642
hello 643
hello 644
hello 645
hello 646
hello 647
hello 648
hello 649
hello 650
hello 651
hello 652
hello 653
hello 654
hello 655
hello 656
hello 657
hello 658
hello 659
hello 660
hello 661
hello 662
hello 663
hello 664
hello 665
hello 666
hello 667
hello 668
hello 669
hello 670
hello 671
hello 672
hello 673
hello 674
hello 675
hello 676
hello 677
hello 678
hello 679
hello 680
hello 681
hello 682
hello 683
hello 684
hello 685
hello 686
hello 687
hello 688
hello 689
hello 690
hello 691
hello 692
hello 693
hello 694
hello 695
hello 696
hello 697
hello 698
hello 699
hello 700
hello 701
hello 702
hello 703

hello 704
hello 705
hello 706
hello 707
hello 708
hello 709
hello 710
hello 711
hello 712
hello 713
hello 714
hello 715
hello 716
hello 717
hello 718
hello 719
hello 720
hello 721
hello 722
hello 723
hello 724
hello 725
hello 726
hello 727
hello 728
hello 729
hello 730
hello 731
hello 732
hello 733
hello 734
hello 735
hello 736
hello 737
hello 738
hello 739
hello 740
hello 741
hello 742
hello 743
hello 744
hello 745
hello 746
hello 747
hello 748
hello 749
hello 750
hello 751
hello 752
hello 753
hello 754
hello 755
hello 756
hello 757
hello 758
hello 759
hello 760
hello 761
hello 762
hello 763
hello 764
hello 765
hello 766
hello 767
hello 768
hello 769
hello 770
hello 771
hello 772
hello 773
hello 774
hello 775
hello 776
hello 777
hello 778
hello 779
hello 780
hello 781
hello 782
hello 783
hello 784
hello 785
hello 786
hello 787
hello 788
hello 789
hello 790
hello 791
hello 792
hello 793
hello 794
hello 795
hello 796
hello 797
hello 798

hello 799
hello 800
hello 801
hello 802
hello 803
hello 804
hello 805
hello 806
hello 807
hello 808
hello 809
hello 810
hello 811
hello 812
hello 813
hello 814
hello 815
hello 816
hello 817
hello 818
hello 819
hello 820
hello 821
hello 822
hello 823
hello 824
hello 825
hello 826
hello 827
hello 828
hello 829
hello 830
hello 831
hello 832
hello 833
hello 834
hello 835
hello 836
hello 837
hello 838
hello 839
hello 840
hello 841
hello 842
hello 843
hello 844
hello 845
hello 846
hello 847
hello 848
hello 849
hello 850
hello 851
hello 852
hello 853
hello 854
hello 855
hello 856
hello 857
hello 858
hello 859
hello 860
hello 861
hello 862
hello 863
hello 864
hello 865
hello 866
hello 867
hello 868
hello 869
hello 870
hello 871
hello 872
hello 873
hello 874
hello 875
hello 876
hello 877
hello 878
hello 879
hello 880
hello 881
hello 882
hello 883
hello 884
hello 885
hello 886
hello 887
hello 888
hello 889
hello 890
hello 891
hello 892
hello 893

hello 894
hello 895
hello 896
hello 897
hello 898
hello 899
hello 900
hello 901
hello 902
hello 903
hello 904
hello 905
hello 906
hello 907
hello 908
hello 909
hello 910
hello 911
hello 912
hello 913
hello 914
hello 915
hello 916
hello 917
hello 918
hello 919
hello 920
hello 921
hello 922
hello 923
hello 924
hello 925
hello 926
hello 927
hello 928
hello 929
hello 930
hello 931
hello 932
hello 933
hello 934
hello 935
hello 936
hello 937
hello 938
hello 939
hello 940
hello 941
hello 942
hello 943
hello 944
hello 945
hello 946
hello 947
hello 948
hello 949
hello 950
hello 951
hello 952
hello 953
hello 954
hello 955
hello 956
hello 957
hello 958
hello 959
hello 960
hello 961
hello 962
hello 963
hello 964
hello 965
hello 966
hello 967
hello 968
hello 969
hello 970
hello 971
hello 972
hello 973
hello 974
hello 975
hello 976
hello 977
hello 978
hello 979
hello 980
hello 981
hello 982
hello 983
hello 984
hello 985
hello 986
hello 987
hello 988

hello 989
hello 990
hello 991
hello 992
hello 993
hello 994
hello 995
hello 996
hello 997
hello 998
hello 999
hello 1000
hello 1001
hello 1002
hello 1003
hello 1004
hello 1005
hello 1006
hello 1007
hello 1008
hello 1009
hello 1010
hello 1011
hello 1012
hello 1013
hello 1014
hello 1015
hello 1016
hello 1017
hello 1018
hello 1019
hello 1020
hello 1021
hello 1022
hello 1023
hello 1024
hello 1025
hello 1026
hello 1027
hello 1028
hello 1029
hello 1030
hello 1031
hello 1032
hello 1033
hello 1034
hello 1035
hello 1036
hello 1037
hello 1038
hello 1039
hello 1040
hello 1041
hello 1042
hello 1043
hello 1044
hello 1045
hello 1046
hello 1047
hello 1048
hello 1049
hello 1050
hello 1051
hello 1052
hello 1053
hello 1054
hello 1055
hello 1056
hello 1057
hello 1058
hello 1059
hello 1060
hello 1061
hello 1062
hello 1063
hello 1064
hello 1065
hello 1066
hello 1067
hello 1068
hello 1069
hello 1070
hello 1071
hello 1072
hello 1073
hello 1074
hello 1075
hello 1076
hello 1077
hello 1078
hello 1079
hello 1080
hello 1081
hello 1082
hello 1083

hello 1084
hello 1085
hello 1086
hello 1087
hello 1088
hello 1089
hello 1090
hello 1091
hello 1092
hello 1093
hello 1094
hello 1095
hello 1096
hello 1097
hello 1098
hello 1099
hello 1100
hello 1101
hello 1102
hello 1103
hello 1104
hello 1105
hello 1106
hello 1107
hello 1108
hello 1109
hello 1110
hello 1111
hello 1112
hello 1113
hello 1114
hello 1115
hello 1116
hello 1117
hello 1118
hello 1119
hello 1120
hello 1121
hello 1122
hello 1123
hello 1124
hello 1125
hello 1126
hello 1127
hello 1128
hello 1129
hello 1130
hello 1131
hello 1132
hello 1133
hello 1134
hello 1135
hello 1136
hello 1137
hello 1138
hello 1139
hello 1140
hello 1141
hello 1142
hello 1143
hello 1144
hello 1145
hello 1146
hello 1147
hello 1148
hello 1149
hello 1150
hello 1151
hello 1152
hello 1153
hello 1154
hello 1155
hello 1156
hello 1157
hello 1158
hello 1159
hello 1160
hello 1161
hello 1162
hello 1163
hello 1164
hello 1165
hello 1166
hello 1167
hello 1168
hello 1169
hello 1170
hello 1171
hello 1172
hello 1173
hello 1174
hello 1175
hello 1176
hello 1177
hello 1178

hello 1179
hello 1180
hello 1181
hello 1182
hello 1183
hello 1184
hello 1185
hello 1186
hello 1187
hello 1188
hello 1189
hello 1190
hello 1191
hello 1192
hello 1193
hello 1194
hello 1195
hello 1196
hello 1197
hello 1198
hello 1199
hello 1200
hello 1201
hello 1202
hello 1203
hello 1204
hello 1205
hello 1206
hello 1207
hello 1208
hello 1209
hello 1210
hello 1211
hello 1212
hello 1213
hello 1214
hello 1215
hello 1216
hello 1217
hello 1218
hello 1219
hello 1220
hello 1221
hello 1222
hello 1223
hello 1224
hello 1225
hello 1226
hello 1227
hello 1228
hello 1229
hello 1230
hello 1231
hello 1232
hello 1233
hello 1234
hello 1235
hello 1236
hello 1237
hello 1238
hello 1239
hello 1240
hello 1241
hello 1242
hello 1243
hello 1244
hello 1245
hello 1246
hello 1247
hello 1248
hello 1249
hello 1250
hello 1251
hello 1252
hello 1253
hello 1254
hello 1255
hello 1256
hello 1257
hello 1258
hello 1259
hello 1260
hello 1261
hello 1262
hello 1263
hello 1264
hello 1265
hello 1266
hello 1267
hello 1268
hello 1269
hello 1270
hello 1271
hello 1272
hello 1273

hello 1274
hello 1275
hello 1276
hello 1277
hello 1278
hello 1279
hello 1280
hello 1281
hello 1282
hello 1283
hello 1284
hello 1285
hello 1286
hello 1287
hello 1288
hello 1289
hello 1290
hello 1291
hello 1292
hello 1293
hello 1294
hello 1295
hello 1296
hello 1297
hello 1298
hello 1299
hello 1300
hello 1301
hello 1302
hello 1303
hello 1304
hello 1305
hello 1306
hello 1307
hello 1308
hello 1309
hello 1310
hello 1311
hello 1312
hello 1313
hello 1314
hello 1315
hello 1316
hello 1317
hello 1318
hello 1319
hello 1320
hello 1321
hello 1322
hello 1323
hello 1324
hello 1325
hello 1326
hello 1327
hello 1328
hello 1329
hello 1330
hello 1331
hello 1332
hello 1333
hello 1334
hello 1335
hello 1336
hello 1337
hello 1338
hello 1339
hello 1340
hello 1341
hello 1342
hello 1343
hello 1344
hello 1345
hello 1346
hello 1347
hello 1348
hello 1349
hello 1350
hello 1351
hello 1352
hello 1353
hello 1354
hello 1355
hello 1356
hello 1357
hello 1358
hello 1359
hello 1360
hello 1361
hello 1362
hello 1363
hello 1364
hello 1365
hello 1366
hello 1367
hello 1368

hello 1369
hello 1370
hello 1371
hello 1372
hello 1373
hello 1374
hello 1375
hello 1376
hello 1377
hello 1378
hello 1379
hello 1380
hello 1381
hello 1382
hello 1383
hello 1384
hello 1385
hello 1386
hello 1387
hello 1388
hello 1389
hello 1390
hello 1391
hello 1392
hello 1393
hello 1394
hello 1395
hello 1396
hello 1397
hello 1398
hello 1399
hello 1400
hello 1401
hello 1402
hello 1403
hello 1404
hello 1405
hello 1406
hello 1407
hello 1408
hello 1409
hello 1410
hello 1411
hello 1412
hello 1413
hello 1414
hello 1415
hello 1416
hello 1417
hello 1418
hello 1419
hello 1420
hello 1421
hello 1422
hello 1423
hello 1424
hello 1425
hello 1426
hello 1427
hello 1428
hello 1429
hello 1430
hello 1431
hello 1432
hello 1433
hello 1434
hello 1435
hello 1436
hello 1437
hello 1438
hello 1439
hello 1440
hello 1441
hello 1442
hello 1443
hello 1444
hello 1445
hello 1446
hello 1447
hello 1448
hello 1449
hello 1450
hello 1451
hello 1452
hello 1453
hello 1454
hello 1455
hello 1456
hello 1457
hello 1458
hello 1459
hello 1460
hello 1461
hello 1462
hello 1463

hello 1464
hello 1465
hello 1466
hello 1467
hello 1468
hello 1469
hello 1470
hello 1471
hello 1472
hello 1473
hello 1474
hello 1475
hello 1476
hello 1477
hello 1478
hello 1479
hello 1480
hello 1481
hello 1482
hello 1483
hello 1484
hello 1485
hello 1486
hello 1487
hello 1488
hello 1489
hello 1490
hello 1491
hello 1492
hello 1493
hello 1494
hello 1495
hello 1496
hello 1497
hello 1498
hello 1499
hello 1500
hello 1501
hello 1502
hello 1503
hello 1504
hello 1505
hello 1506
hello 1507
hello 1508
hello 1509
hello 1510
hello 1511
hello 1512
hello 1513
hello 1514
hello 1515
hello 1516
hello 1517
hello 1518
hello 1519
hello 1520
hello 1521
hello 1522
hello 1523
hello 1524
hello 1525
hello 1526
hello 1527
hello 1528
hello 1529
hello 1530
hello 1531
hello 1532
hello 1533
hello 1534
hello 1535
hello 1536
hello 1537
hello 1538
hello 1539
hello 1540
hello 1541
hello 1542
hello 1543
hello 1544
hello 1545
hello 1546
hello 1547
hello 1548
hello 1549
hello 1550
hello 1551
hello 1552
hello 1553
hello 1554
hello 1555
hello 1556
hello 1557
hello 1558

hello 1559
hello 1560
hello 1561
hello 1562
hello 1563
hello 1564
hello 1565
hello 1566
hello 1567
hello 1568
hello 1569
hello 1570
hello 1571
hello 1572
hello 1573
hello 1574
hello 1575
hello 1576
hello 1577
hello 1578
hello 1579
hello 1580
hello 1581
hello 1582
hello 1583
hello 1584
hello 1585
hello 1586
hello 1587
hello 1588
hello 1589
hello 1590
hello 1591
hello 1592
hello 1593
hello 1594
hello 1595
hello 1596
hello 1597
hello 1598
hello 1599
hello 1600
hello 1601
hello 1602
hello 1603
hello 1604
hello 1605
hello 1606
hello 1607
hello 1608
hello 1609
hello 1610
hello 1611
hello 1612
hello 1613
hello 1614
hello 1615
hello 1616
hello 1617
hello 1618
hello 1619
hello 1620
hello 1621
hello 1622
hello 1623
hello 1624
hello 1625
hello 1626
hello 1627
hello 1628
hello 1629
hello 1630
hello 1631
hello 1632
hello 1633
hello 1634
hello 1635
hello 1636
hello 1637
hello 1638
hello 1639
hello 1640
hello 1641
hello 1642
hello 1643
hello 1644
hello 1645
hello 1646
hello 1647
hello 1648
hello 1649
hello 1650
hello 1651
hello 1652
hello 1653

hello 1654
hello 1655
hello 1656
hello 1657
hello 1658
hello 1659
hello 1660
hello 1661
hello 1662
hello 1663
hello 1664
hello 1665
hello 1666
hello 1667
hello 1668
hello 1669
hello 1670
hello 1671
hello 1672
hello 1673
hello 1674
hello 1675
hello 1676
hello 1677
hello 1678
hello 1679
hello 1680
hello 1681
hello 1682
hello 1683
hello 1684
hello 1685
hello 1686
hello 1687
hello 1688
hello 1689
hello 1690
hello 1691
hello 1692
hello 1693
hello 1694
hello 1695
hello 1696
hello 1697
hello 1698
hello 1699
hello 1700
hello 1701
hello 1702
hello 1703
hello 1704
hello 1705
hello 1706
hello 1707
hello 1708
hello 1709
hello 1710
hello 1711
hello 1712
hello 1713
hello 1714
hello 1715
hello 1716
hello 1717
hello 1718
hello 1719
hello 1720
hello 1721
hello 1722
hello 1723
hello 1724
hello 1725
hello 1726
hello 1727
hello 1728
hello 1729
hello 1730
hello 1731
hello 1732
hello 1733
hello 1734
hello 1735
hello 1736
hello 1737
hello 1738
hello 1739
hello 1740
hello 1741
hello 1742
hello 1743
hello 1744
hello 1745
hello 1746

hello 1747

hello 1748
hello 1749
hello 1750
hello 1751
hello 1752
hello 1753
hello 1754
hello 1755
hello 1756
hello 1757
hello 1758
hello 1759
hello 1760
hello 1761
hello 1762
hello 1763
hello 1764
hello 1765
hello 1766
hello 1767
hello 1768
hello 1769
hello 1770
hello 1771
hello 1772
hello 1773
hello 1774
hello 1775
hello 1776
hello 1777
hello 1778
hello 1779
hello 1780
hello 1781
hello 1782
hello 1783
hello 1784
hello 1785
hello 1786
hello 1787
hello 1788
hello 1789
hello 1790
hello 1791
hello 1792
hello 1793
hello 1794
hello 1795
hello 1796
hello 1797
hello 1798
hello 1799
hello 1800
hello 1801
hello 1802
hello 1803
hello 1804
hello 1805
hello 1806
hello 1807
hello 1808
hello 1809
hello 1810
hello 1811
hello 1812
hello 1813
hello 1814
hello 1815
hello 1816
hello 1817
hello 1818
hello 1819
hello 1820
hello 1821
hello 1822
hello 1823
hello 1824
hello 1825
hello 1826
hello 1827
hello 1828
hello 1829
hello 1830
hello 1831
hello 1832
hello 1833
hello 1834
hello 1835
hello 1836
hello 1837
hello 1838
hello 1839
hello 1840
hello 1841
hello 1842

hello 1843
hello 1844
hello 1845
hello 1846
hello 1847
hello 1848
hello 1849
hello 1850
hello 1851
hello 1852
hello 1853
hello 1854
hello 1855
hello 1856
hello 1857
hello 1858
hello 1859
hello 1860
hello 1861
hello 1862
hello 1863
hello 1864
hello 1865
hello 1866
hello 1867
hello 1868
hello 1869
hello 1870
hello 1871
hello 1872
hello 1873
hello 1874
hello 1875
hello 1876
hello 1877
hello 1878
hello 1879
hello 1880
hello 1881
hello 1882
hello 1883
hello 1884
hello 1885
hello 1886
hello 1887
hello 1888
hello 1889
hello 1890
hello 1891
hello 1892
hello 1893
hello 1894
hello 1895
hello 1896
hello 1897
hello 1898
hello 1899
hello 1900
hello 1901
hello 1902
hello 1903
hello 1904
hello 1905
hello 1906
hello 1907
hello 1908
hello 1909
hello 1910
hello 1911
hello 1912
hello 1913
hello 1914
hello 1915
hello 1916
hello 1917
hello 1918
hello 1919
hello 1920
hello 1921
hello 1922
hello 1923
hello 1924
hello 1925
hello 1926
hello 1927
hello 1928
hello 1929
hello 1930
hello 1931
hello 1932
hello 1933
hello 1934
hello 1935
hello 1936
hello 1937

```
hello 1938
hello 1939
hello 1940
hello 1941
hello 1942
hello 1943
hello 1944
hello 1945
hello 1946
hello 1947
hello 1948
hello 1949
hello 1950
hello 1951
hello 1952
hello 1953
hello 1954
hello 1955
hello 1956
hello 1957
hello 1958
hello 1959
hello 1960
hello 1961
hello 1962
hello 1963
hello 1964
hello 1965
hello 1966
hello 1967
```

```
-----
RecursionError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_4872\1209717682.py in <module>
      9     great()
     10
--> 11 great()

~\AppData\Local\Temp\ipykernel_4872\1209717682.py in great()
      7     i=i+1
      8     print('hello',i)
----> 9     great()
     10
     11 great()

... last 1 frames repeated, from the frame below ...

~\AppData\Local\Temp\ipykernel_4872\1209717682.py in great()
      7     i=i+1
      8     print('hello',i)
----> 9     great()
     10
     11 great()

RecursionError: maximum recursion depth exceeded while calling a Python object
```

In [23]

```
def fact(n):
    if n==0:
        return 1
    return n*fact(n-1)
result=fact(5)
print(result)
```

In [24]

```
#Anonymus function(function without name)
f1= lambda a: a*a
result1= f1(5)
print(result1)
f2= lambda a,b: a+b
result2= f2(5,9)
print(result2)
```

Out

```
25
14
```

In [25]

```
#def is_even(n):  
    #return n%2==0  
from functools import reduce  
num=[2,4,8,5,7,43]  
evens=list(filter(lambda n: n%2==0, num))  
print(evens)  
doubles=list(map(lambda n: n*2, num))  
print(doubles)  
red=reduce(lambda a,b: a+b , doubles)  
print(red)
```

Out

```
[2, 4, 8]  
[4, 8, 16, 10, 14, 86]  
138
```

In [26]

```
def div(a,b):# decorator  
    print(a/b)  
def smart_div(func):  
    def inner(a,b):  
        if a<b:  
            a,b=b,a  
        return (func(a,b))  
    return inner  
div =smart_div(div)  
div(2,4)
```

Out

```
2.0
```

In [27]

```
print(__name__)
```

Out

```
__main__
```

In [28]

```
print ('hello'+ __name__)
```

Out

```
hello__main__
```

In [29]

```
print ('demosays'+ __name__)
```

Out

```
demosays__main__
```

In [1]

```
X = [[8,5],
      [9 ,3],
      [4 ,6]]
Y = [[8,5],
      [9,5],
      [9,4]]
result = [[0,0],
           [0,0],
           [0,0]]
print(len(X))
print(len(X[0]))
for i in range(len(X)):
    for j in range(len(X[0])):
        result[i][j] = X[i][j] + Y[i][j]
for k in result:
    print(k)
```

Out

```
3
2
[16, 10]
[18, 8]
[13, 10]
```

In [2]

```
X = [[8,5,1],
      [9 ,3,2],
      [4 ,6,3]]
Y = [[8,5,3],
      [9,5,7],
      [9,4,1]]
result = [[0,0,0,0],
           [0,0,0,0],
           [0,0,0,0]]
print(len(X))
print(len(Y))
print(len(Y[0]))
for i in range(len(X)):
    for j in range(len(Y[0])):
        for k in range(len(Y)):
            result[i][j] += X[i][k] * Y[k][j]
for x in result:
    print(x)
```

Out

```
3
3
3
[118, 69, 60, 0]
[117, 68, 50, 0]
[113, 62, 57, 0]
```

sql.ipynb

In [1]

```
#python to connect database
#mysql installer (download and install)
#developer default
'''
create database fahad;

use fahad;
create table student (name varchar(20), college varchar(20));
insert into student values ('faisal','nit'),('ashish','iit');
select * from student;
'''
```

Out

```
"\ncreate database fahad;\n\nuse fahad;\ncreate table student (name varchar(20), college varchar(20));\ninsert into student values ('faisal','nit'),('ashish','iit');\nselect * from student;\n"
```

In [2]

```
#python database connection
```



```
# -*- coding: utf-8 -*-
"""
Created on Wed Feb 23 08:49:56 2022

@author: 91997
"""

def main():
    print("its time to calculate")
if __name__ == "__main__":
    main()
```

```
# -*- coding: utf-8 -*-
"""
Created on Mon Feb 21 08:42:03 2022

@author: 91997
"""

def add (a,b):
    return a+b
def sub (a,b):
    return a-b
def Mul (a,b):
    return a*b
def div (a,b):
    return a/b
```

client.py

```
# -*- coding: utf-8 -*-
"""
Created on Thu Feb 17 09:49:00 2022

@author: 91997
"""

import socket
s=socket.socket()
print('socket created')
s.bind(('localhost', 9999))
s.listen(3)

print ('waiting for connection')

while True:
    c,addr=s.accept()
    print('connected with ', addr)
    c.send('welcome to vit')
    c.close
```

dem.py

```
# -*- coding: utf-8 -*-
"""
Created on Wed Feb 23 08:51:30 2022

@author: 91997
"""
import cal
print(__name__)
print("hello")
print ("welcome user")
```

server.py

```
# -*- coding: utf-8 -*-
"""
Created on Thu Feb 17 09:49:00 2022

@author: 91997
"""

import socket
s=socket.socket()
#ipv/iv6, tcp or udp  default ipv4 and udp
print('socket created')
#socket created
#bind socket with port number
#ipaddress, port number
s.bind(('localhost', 9999))
#listent to client (3 clients waiting for connection)
s.listen(3)

print ('waiting for connection')

while True:
    # accept the client request and process
    #c-socket and address of the client
    c,addr=s.accept()
    #name of the client
    #name =c.recv(1024).decode()
    print('connected with ', addr)
    #send message to the client
    c.send(bytes('Welcome to VIT','utf-8'))
    c.close
```

```
# -*- coding: utf-8 -*-
"""
Created on Wed Feb 16 15:16:28 2022

@author: 91997
"""

import mysql.connector
mydb=mysql.connector.connect(host="localhost", user="root", password="12345", database='fahad')
mycursor=mydb.cursor()
#mycursor.execute("show databases")
mycursor.execute("select * from student")
#result=mycursor.fetchall()
result=mycursor.fetchone()
for i in result:
    print(i)
#for i in mycursor:
    #print(i)
```

Images

new.png

```
main() {  
  
    static int b=3;  
  
    if(b)  
    for (int i=1; i<=3; i++)  
    {  
        printf("%d", b--);  
    }  
  
}
```

new1.png

```
int main() {  
    for (int i=0; i<10; ++i)  
    {  
        printf("%d", i<<1);  
    }  
  
    return 0;  
}
```

q13.png

```
int main() {  
    for (int i=0; i<5; ++i)  
    {  
        printf("%d", i<<2);  
    }  
  
    return 0;  
}
```

q15.png

```

int main() {
    int x=10, y=20;
    int *p, *q, *r;
    p=&x;
    q=p;
    r=q;
    printf("%d%d%d%d", x, *p, *q, *r,y);

    return 0;
}

```

q8.jpeg

```

#include<stdio.h>
int i;
int main()
]{
    int n;
    if(i)
        n=6;
    else
        n=9;
    for (; n; )
        printf(" %d", n--);
    return 0;
}

```

q9.jpeg

```

#include <stdio.h>

int main()
{
    int a,b;
    a = (0,1);
    b = 0,1;
    int c = a && --b;
    int d = --a || --b;
    printf("a = %d, b = %d, c = %d, d = %d", a, b, c, d);
    return 0;
}

```